11장

프로젝트로 파이썬 완성하기: 핫플레이스 원쿠 분석

지금까지 배운 내용을 바탕으로 실제 파이썬 프로젝트를 진행해 보겠습니다. 프로젝트 주제에 따라 프로젝트 목표를 수립하고 프로 그램을 구현한 후 실행결과를 분석합니다. 11장 2 절 최 부분에서 원쿠했듯이 핫플레이스 원쿠 데이터를 분석합니다. 유명한 맛집이나 카페 등에는 언제 가야 가장 덜 붐빌지 원쿠 분석을 통해 사람이 적은 시간대를 구해 보겠습니다.

11.1

프로젝트 목표 수립하기

10장에서 배운 대로 첫 단계인 데이터를 선정해 수집하고 프로젝트 목표를 수립해 봅시다. 여기서는 핫플레이스 인구를 표현할 수 있는 데이터를 수집한 후, 수집한 데이터를 바탕으로 목표를 수립해 보겠습니다.

11.1.1 데이터 선정하기

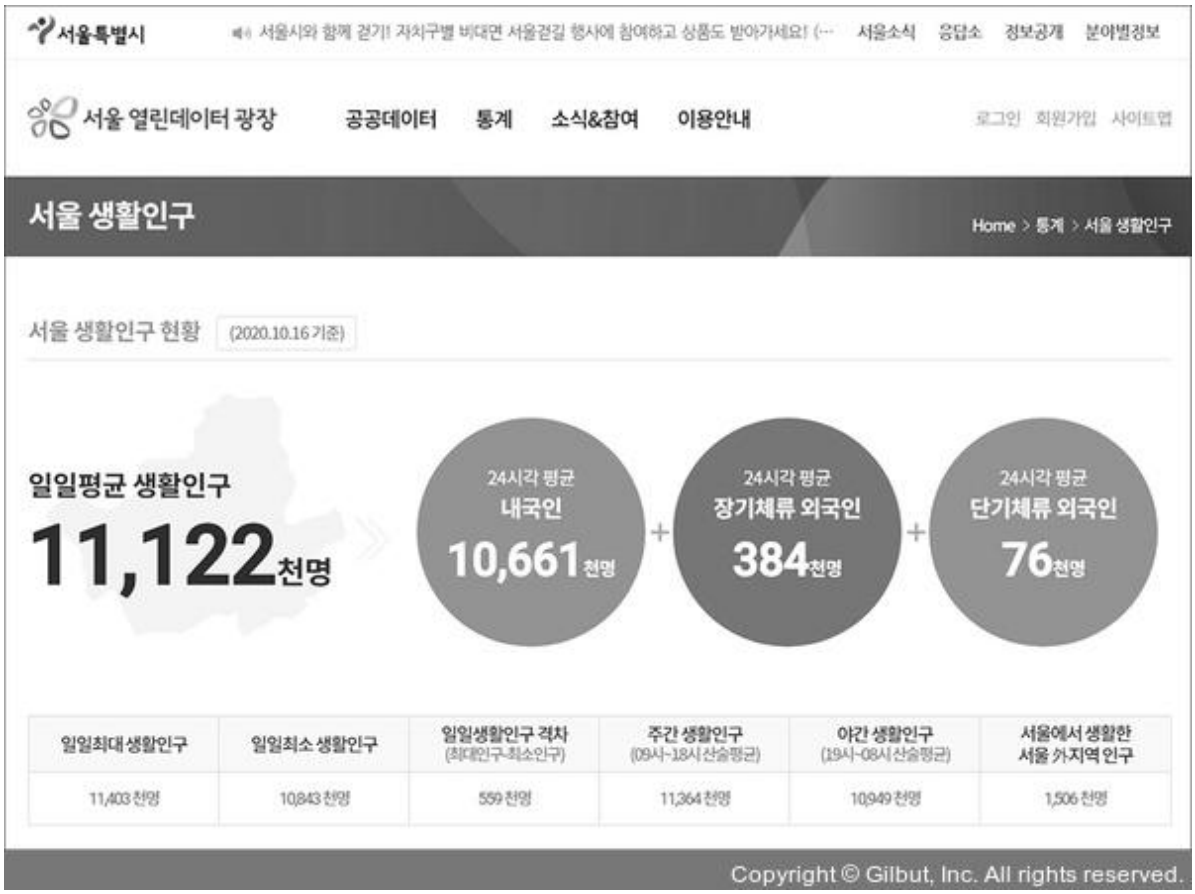
핫플레이스의 인구 분석이므로 인구 데이터가 필요하겠네요. 그리고 사람이 적은 시간대를 구하니 시간대별 인구 데이터가 있어야 하고요. 그런데 찾는 범위를 전국으로 하면 데이터가 너무 커질 테니 지역을 서울로 한정하겠습니다.

데이터를 찾을 때는 구글을 추천합니다. 찾으려는 키워드 옆에 '데이터'를 붙여 구글에서 검색하면 적절한 데이터를 찾을 수 있습니다. 우리에게 필요한 데이터는 서울의 시간대별 인구죠. 그럼 구글에서 '서울 시간대별 인구 데이터'를 검색해 봅시다(다른 지역을 알고 싶다면 해당 지역으로 바꿔서 검색하면 됩니다).



▲ 그림 11-1 구글에서 데이터 찾기

검색 결과가 뜨면 하나씩 클릭하면서 찾는 데이터가 맞는지 확인합니다. 첫 번째로 나온 서울 생활인구> 데이터즐기기 | 서울열린데이터광장을 클릭해 봅시다.



▲ 그림 11-2 서울 생활인구 페이지

'서울 열린데이터 광장'이라는 사이트의 서울 생활인구 페이지가 뜨네요. '생활인구'라는 단어가 조금 생소한데, 생활인구란 거주인구가 아닌 실제로 생활하는 인구로, 통신기지국을 기반으로 1시간 단위로 추정한 인구 데이터입니다(이 책에서 인구는 생활인구를 의미합니다).

페이지를 좀 내려 보면 중간쯤에 데이터 내려받기가 있습니다. 집계구 단위, 행정동 단위, 자치구 단위로 나뉘어 있는데, 용어가 다소 생소합니다. 주소는 '서울시 00구 00동'으로 표현하죠. 여기에서 '00구'에 해당하는 것이 자치구입니다. 서울에는 총 25개 자치구가 있습니다. 그다음 00동에 해당하는 것이 행정동입니다. 행정동은 자치구를 한 번 더 하위 지역을 나눈 것으로, 강남구를 보면 개포동, 논현동, 도곡동, 삼성동 등의 행정동이 있죠. 서울에는 총 424개의 행정동이 있습니다. 당연히 행정동이 자치구보다 더 세밀하게 지역을 분석할 수 있겠죠? 그래서 행정동 데이터로 데이터 분석 프로젝트를 진행하겠습니다.

그럼 데이터를 내려받아 봅시다. 행정동 단위에서 [서울 생활인구 (내국인)]을 클릭합니다.



▲ 그림 11-3 데이터 내려받기

NOTE 집계구가 아닌 행정동 단위를 선택한 이유

집계구는 인구 규모(최적 500명), 사회경제적 동질성(주택유형, 평균지가), 집계구의 형태와 크기를 고려해 행정동을 더 세부적으로 나눈 통계 단위입니다. 서울 생활인구 페이지에 따르면 2020년 12월 기준 서울에는 총 19,153개의 집계구가 있습니다. 집계구 숫자가 압도적으로 많으니 집계구 데이터로 더 정확한 분석이 가능하다고 생각할 수 있어요. 그러나 집계구 데이터를 확인해 보면 해당 집계구가 속한 행정동코드와 집계구코드만 나와 있을 뿐이고, 집계구의 위치가 정확하게 어디인지는 없습니다. 그래서 분석하려는 지역을 찾기도, 자료를 보고 분석하기도 어렵습니다.

파일 내려받기 항목에 몇 개 파일만 뜨는데, 파일 목록 아래에 있는 [전체 파일보기]를 클릭하면 전체 파일 목록이 나열됩니다.



▲ 그림 11-4 파일 내려받기

파일명을 잘 보면 마지막에 연도-월이 있습니다. 한 달을 기준으로 데이터가 올라와 있네요. 모든 데이터로 분석하는 것이 원래 취지에 맞으나 규모가 너무 커지므로 파일 하나만으로 진행하겠습니다. 2019년 12월 데이터인 LOCAL_PEOPLE_DONG_201912.zip을 클릭해 파일을 내려받아 봅시다(가장 최근 데이터를 분석하는 것이 좋지만 2020년 1월부터 코로나19 때문에 인구 이동이 감소됐죠. 그래서 2019년 12월 데이터로 진행합니다).

파일을 내려받고 압축을 풀면 폴더 안에 LOCAL_PEOPLE_DONG_201912.csv 파일이 있습니다. 파일을 마우스로 우클릭한 후 연결 프로그램을 Excel로 선택해 열어 보세요.

	A	B	C	D	E	F	G	H	I
1	기준일ID	시간대구분	행정동코드	총생활인구수	남자0세부터9	남자10세부터	남자15세부터	남자20세부터	남자25세부터
2	20191201	0	11215710	45694.9988	452.3937	208.7971	1267.7678	5642.9235	5433.6801
3	20191201	0	11680545	37375.4506	1298.5585	731.2901	889.5126	895.5983	1397.2099
4	20191201	0	11680610	37266.9526	1083.215	1559.2566	2339.8178	991.4434	990.1854
5	20191201	0	11680655	25231.2064	1191.0306	607.5946	639.2233	608.4116	697.5009
6	20191201	0	11710570	40728.5564	1586.6273	818.9047	1204.9034	1172.2571	1195.7198
7	20191201	0	11710531	11547.1842	310.5995	219.2461	343.8328	321.8387	435.1919
8	20191201	0	11710561	16811.9601	731.3241	448.4862	746.4326	397.8013	509.9872
9	20191201	0	11200535	23920.0334	932.6176	269.9108	404.7168	749.2405	1078.3448
10	20191201	0	11710690	25047.6622	1931.6501	756.8446	444.8957	424.1559	611.9113

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 11-5 내려받은 서울 생활인구 데이터 파일

굉장히 많은 데이터가 있네요. 첫 번째 행에 있는 헤더를 살펴봅시다.

기준일ID	시간대구분	행정동코드	총생활인구수	남자0세부터9세생활인구수	남자10세부터14세생활인구수	남자15세부터19세생활인구수	남자20세부터24세생활인구수	남자25세부터29세생활인구수	남자30세부터34세생활인구수	남자35세부터39세생활인구수	남자40세부터44세생활인구수	남자45세부터49세생활인구수	남자50세부터54세생활인구수	남자55세부터59세생활인구수	남자60세부터64세생활인구수	남자65세부터69세생활인구수	남자70세이상생활인구수	여자0세부터9세생활인구수	여자10세부터14세생활인구수	여자15세부터19세생활인구수	여자20세부터24세생활인구수	여자25세부터29세생활인구수	여자30세부터34세생활인구수	여자35세부터39세생활인구수	여자40세부터44세생활인구수	여자45세부터49세생활인구수	여자50세부터54세생활인구수	여자55세부터59세생활인구수	여자60세부터64세생활인구수	여자65세부터69세생활인구수	여자70세이상생활인구수

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 11-6 서울 생활인구 데이터 파일의 헤더

총 32개의 항목이 있습니다. 기준일ID와 시간대구분, 행정동코드, 총 생활인구수가 있고, 남자 연령대별 인구수와 여자 연령대별 인구수가 차례대로 있습니다. 성별, 연령대별 인구수는 나중에 확인하고 먼저 앞의 4개 항목만 보겠습니다.

	A	B	C	D
1	기준일ID	시간대구분	행정동코드	총생활인구수
2	20191201	0	11215710	45694.9988
3	20191201	0	11680545	37375.4506
4	20191201	0	11680610	37266.9526
5	20191201	0	11680655	25231.2064

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 11-7 데이터 파일의 일부 항목

헤더 다음의 첫 번째 행을 해석하면 2019년 12월 1일 0시에 행정동코드 11215710인 지역의 총 생활인구가 45694.9988명이라는 의미입니다. 인구가 소수점으로 나오네요. 서울 열린데이터 광장에서 제공하는 생활인구 매뉴얼을 잘 살펴보면 인구는 통계 추정치이기 때문에 소수점이 발생할 수 있다고 되어 있습니다. 두 번째 행은 2019년 12월 1일 0시에 행정동코드 11680545인 지역의 총 생활인구가 37375.4506명임을 나타냅니다.

그런데 데이터에는 행정동코드 8자리만 나와 있고 실제 어느 지역인지를 알 수가 없어요. 행정동코드 11215710이 어떤 동인지를 알아야 프로젝트를 진행하기 쉬울 텐데 말이죠. 이때 참고할 자료가 있습니다. 서울 열린데이터 광장에서는 행정동코드와 코드에 해당하는 행정동명을 별도의 파일로 제공합니다. 서울 생활인구 페이지 (https://data.seoul.go.kr/dataVisual/seoul/seoulLivingPopulation.do)로 다시 가서 이번에는 자치구 단위의 [행정구역 코드정보]를 클릭해 파일을 내려받습니다.



▲ 그림 11-8 행정구역 코드정보 내려받기

내려받은 '행정동코드_매핑정보.xlsl' 파일을 열어 보면 하단에 '행정동코드'와 '유입지코드'라는 2개의 시트가 있습니다. 우리에게 필요한 것은 행정동코드이므로 [행정동코드] 시트를 클릭합니다.

	A	B	C	D	E
1	통계청행정동코드	행자부행정동코드	시도명	시군구명	행정동명
2	H_SDNG_CD	H_DNG_CD	DO_NM	CT_NM	H_DNG_NM
3	1101053	11110530	서울	종로구	사직동
4	1101054	11110540	서울	종로구	삼청동
5	1101055	11110550	서울	종로구	부암동
6	1101056	11110560	서울	종로구	평창동
7	1101057	11110570	서울	종로구	무악동

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 11-9 행정동코드 정보

헤더에 2가지 행정동코드와 시도명, 시군구명, 행정동명이 보입니다. 그런데 통계청 코드는 숫자 7자리, 행자부 코드는 8자리입니다. 앞서 내려받은 LOCAL_PEOPLE_DONG_201912.csv 파일의 행정동코드는 8자리고요. 따라서 여기서는 두 번째 열에 있는 8자리 행자부 행정동코드를 사용하면 됩니다. 행정동코드_매핑정보.xlsl 파일에서 11215710과 11680545를 찾아 보니 광진구 화양동과 강남구 압구정동입니다.

이처럼 LOCAL_PEOPLE_DONG_201912.csv 파일로는 인구 데이터를, 행정동코드_매핑정보.xlsl 파일로는 행정동명을 알 수 있으므로 필요한 데이터는 다음과 같이 선정합니다.

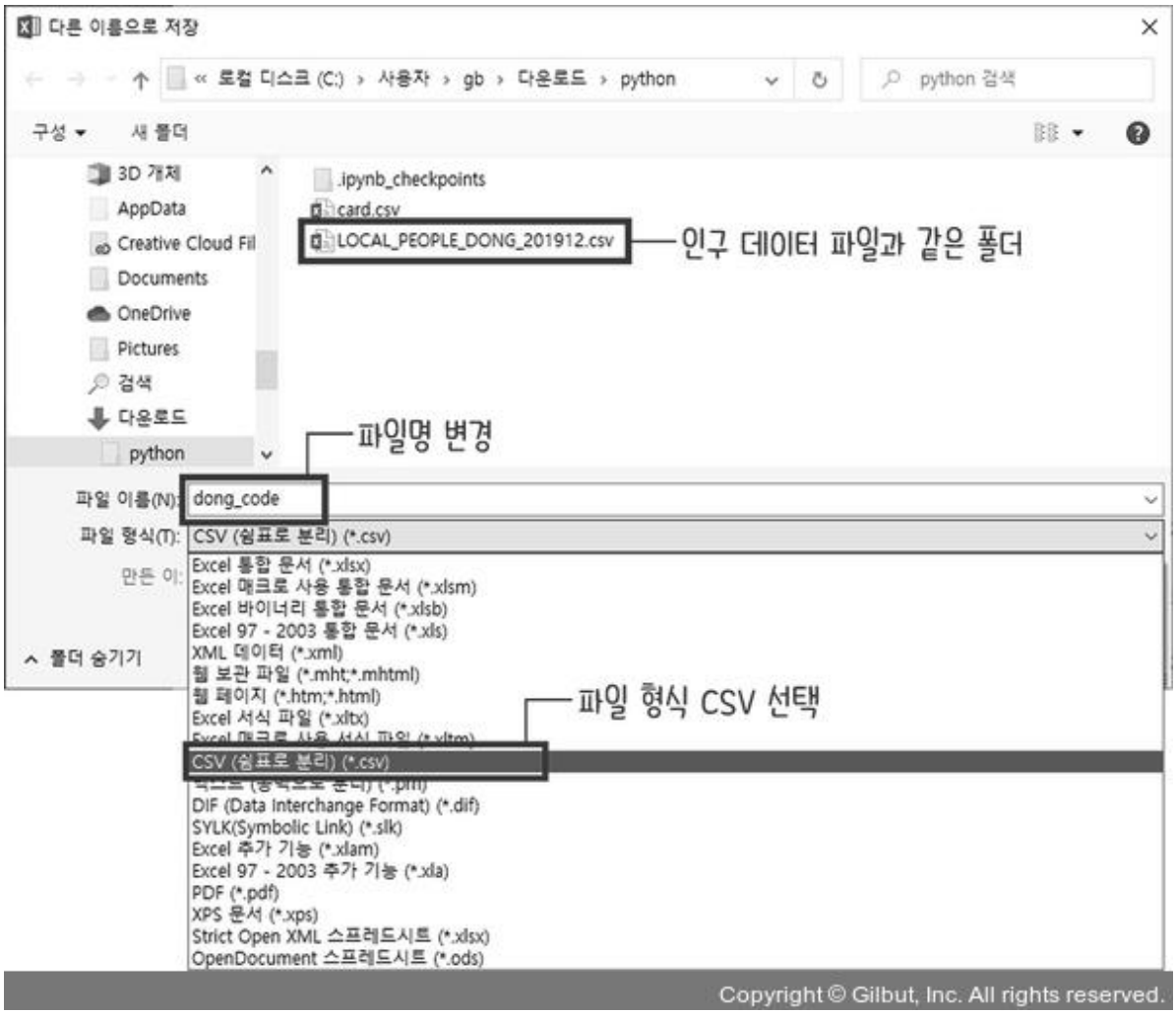
1. 프로젝트 목표 수립

   1) 데이터 선정하기

   데이터

   인구 데이터: 서울 생활인구(내국인)
   행정동코드 데이터: 행정구역 코드정보

파이썬에서 csv 라이브러리를 사용할 수 있도록 내려받은 파일의 파일명과 확장자를 수정하겠습니다. 행정동코드_매핑정보.xlsl 파일을 dong_code.csv로 바꿉니다. 행정동코드_매핑정보.xlsl 파일의 [행정동코드] 시트에서 [파일 → 다른 이름으로 저장]을 누르고 LOCAL_PEOPLE_DONG_201912.csv 파일과 같은 폴더를 선택합니다. 팝업 창이 뜨면 파일명은 dong_code로 하고, 파일 형식은 CSV (쉼표로 분리) (`*`.csv) 또는 CSV UTF-8(쉼표로 분리) (.csv)를 선택합니다. 이때 2개의 시트를 하나의 파일로 만들기 때문에 경고창이 뜹니다. [확인] 버튼을 누르면 현재 열려 있는 시트만 CSV 파일로 변환합니다.





▲ 그림 11-10 파일명과 파일 형식 바꾸기

11.1.2 목표 수립하기

다음으로 프로젝트의 목표를 수립해 봅시다. 우리는 핫플레이스가 덜 붐비는 시간을 알고 싶습니다. 핫플레이스가 혜화동에 있다면 혜화동의 인구가 가장 적은 시간을 찾으면 됩니다. 따라서 프로젝트의 목표는 핫플레이스가 있는 행정동에서 인구가 가장 적은 시간대 파악하기로 정리할 수 있습니다.

2) 목표 수립하기

목표

핫플레이스가 있는 행정동에서 인구가 가장 적은 시간대 파악하기

목표를 달성할 수 있는 하위 목표를 수립해 봅시다. 하위 목표는 더 구체적으로 분석할 요소를 정하는 것이어서 데이터 항목을 보면서 실제로 달성할 수 있는 내용으로 수립하는 것이 좋습니다.

> 제제쌤의 조언
> 행정동보다 더 세부적으로 접근할 수 있다면, 즉 구체적인 가게명이나 주소를 입력하고 이 지역의 시간대별 인구를 구할 수 있다면 더 정확한 답을 구할 수 있습니다. 하지만 더 세부적인 데이터는 구글 검색만으로 구할 수 없습니다. 데이터 분석은 데이터를 구하는 것(수집)에서 시작합니다. 이처럼 원하는 데이터를 구하기 어려울 때는 구할 수 있는 데이터를 활용해 프로젝트 방향을 정해야 합니다.

우리가 가장 궁금한 것은 핫플레이스가 있는 행정동의 시간대별 인구입니다. 행정동의 시간대별 인구는 어떻게 구할까요?

먼저 dong_code.csv 파일에서 핫플레이스가 있는 행정동의 행정동코드를 알아내야죠. LOCAL_PEOPLE_DONG_201912.csv 파일에서 알아낸 행정동코드와 일치하는 행만 골라냅니다. 그리고 시간대별로 나뉜, 네 번째 항목에 있는 생활인구수를 더한 후 평균을 구하면 됩니다.



	A	B	C	D	E
1	통계청행정동코드	행자부행정동코드	시도명	시군구명	행정동명
376	1123075	11680750	서울	강남구	수서동
377	1123076	11680700	서울	강남구	세곡동
378	1123077	11680545	서울	강남구	압구정동
379	1123078	11680565	서울	강남구	청담동
380	1123079	11680610	서울	강남구	대치2동
381	1123080	11680670	서울	강남구	개포2동
382	1124051	11710510	서울	송파구	풍납1동

dong_code.csv

	A	B	C	D
1	기준일ID	시간대구분	행정동코드	총생활인구수
3	20191201	0	11680545	37375.4506
518	20191201	1	11680545	36036.1208
880	20191201	2	11680545	35050.7777
1549	20191201	3	11680545	34224.7169
1776	20191201	4	11680545	33579.9165
2390	20191201	5	11680545	33636.4017
2597	20191201	6	11680545	34352.6946

LOCAL_PEOPLE_DONG_201912.csv

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 11-11 행정동코드로 평균인구 구하기

구한 평균인구를 시간대별 그래프로 표현하면 인구 변화가 시각적으로 더 잘 보이겠죠? 따라서 첫 번째 하위 목표는 핫플레이스가 있는 행정동의 시간대별 평균인구 그래프를 그려 분석하기로 정합니다. 목표와 함께 그래프를 어떻게 그릴지 간단히 스케치합니다. 꺾은선 그래프가 시간대별 추이를 잘 보여 줄 것 같네요.

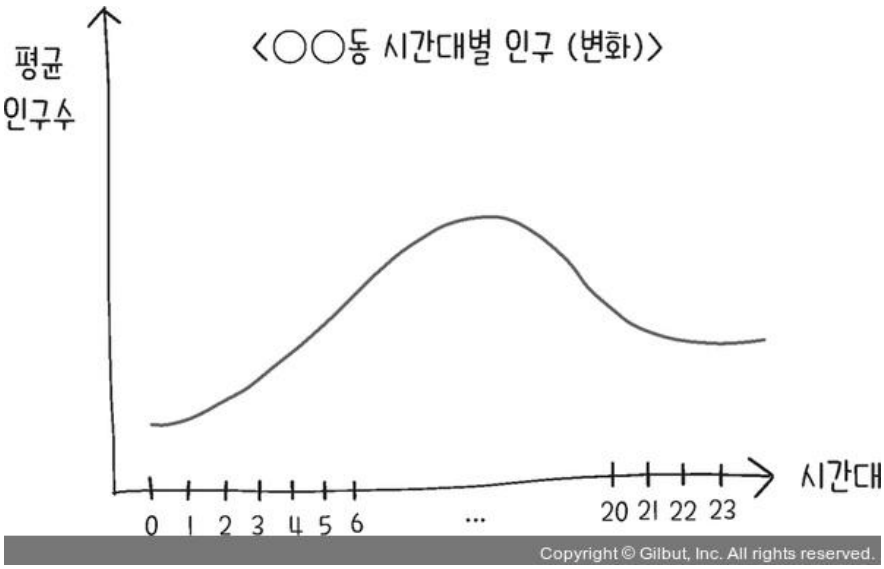
2) 목표 수립

목표

> 핫플레이스가 있는 행정동에서 인구가 가장 적은 시간대 파악하기

하위 목표 1

> 핫플레이스가 있는 행정동의 시간대별 평균인구 그래프를 그려 분석하기



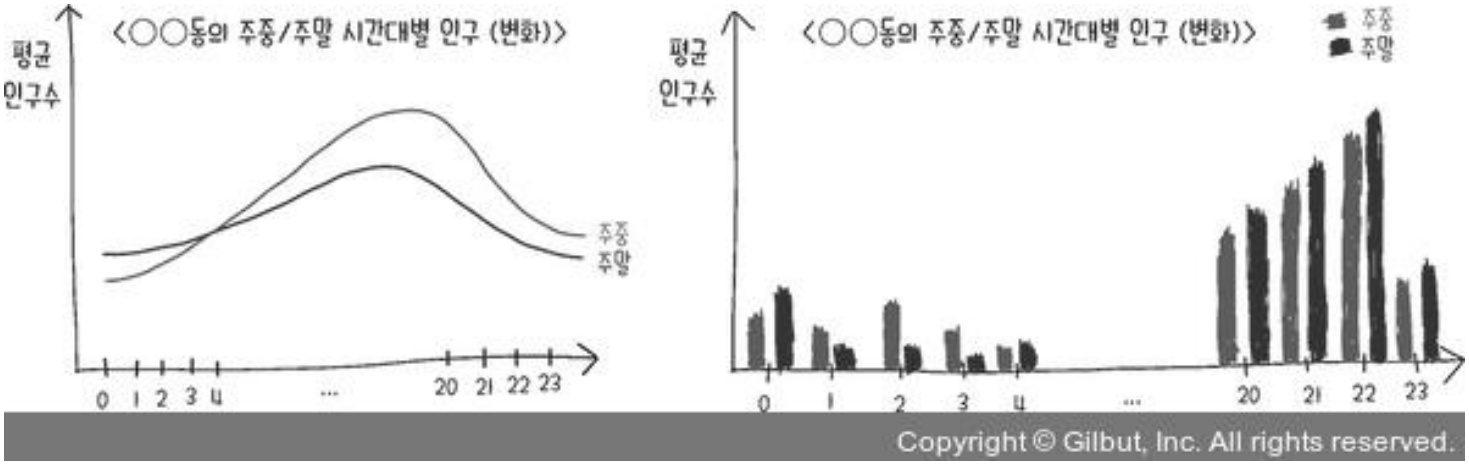
그런데 주중과 주말의 생활인구가 비슷할까요? 놀이공원 입장료나 숙박비나 항공 요금 등을 보면 주말과 주중에 따라 금액을 달리 하는 경우가 많습니다. 이는 사람들이 몰리는 정도가 다르기 때문이죠. 핫플레이스 역시 주중 인구와 주말 인구가 다를 수 있으므로 둘을 나눠서 비교하는 것도 의미 있을 것 같습니다.

데이터 파일은 주중과 주말로 구분되어 있지 않은데, 어떻게 구분할까요? 간단합니다. 데이터 파일을 보면 첫 번째 항목에 일자를 나타내는 기준일ID가 있습니다. 파이썬에는 일자를 입력하면 숫자로 요일을 알려 주는 `datetime` 라이브러리가 있습니다. 이를 활용하면 기준일ID로 요일을 알아내 주중과 주말을 구분할 수 있습니다.

그럼 두 번째 목표는 핫플레이스가 있는 행정동의 주중/주말 시간대별 평균인구 그래프를 그려 분석하기로 하겠습니다. 이 역시 그래프를 미리 스케치합니다. 막대그래프로도 비교 그래프를 그릴 수도 있겠죠. 같은 데이터를 사용하므로 두 그래프 모두 스케치합니다.

하위 목표 2

핫플레이스가 있는 행정동의 주중/주말 시간대별 평균인구 그래프를 그려 분석하기



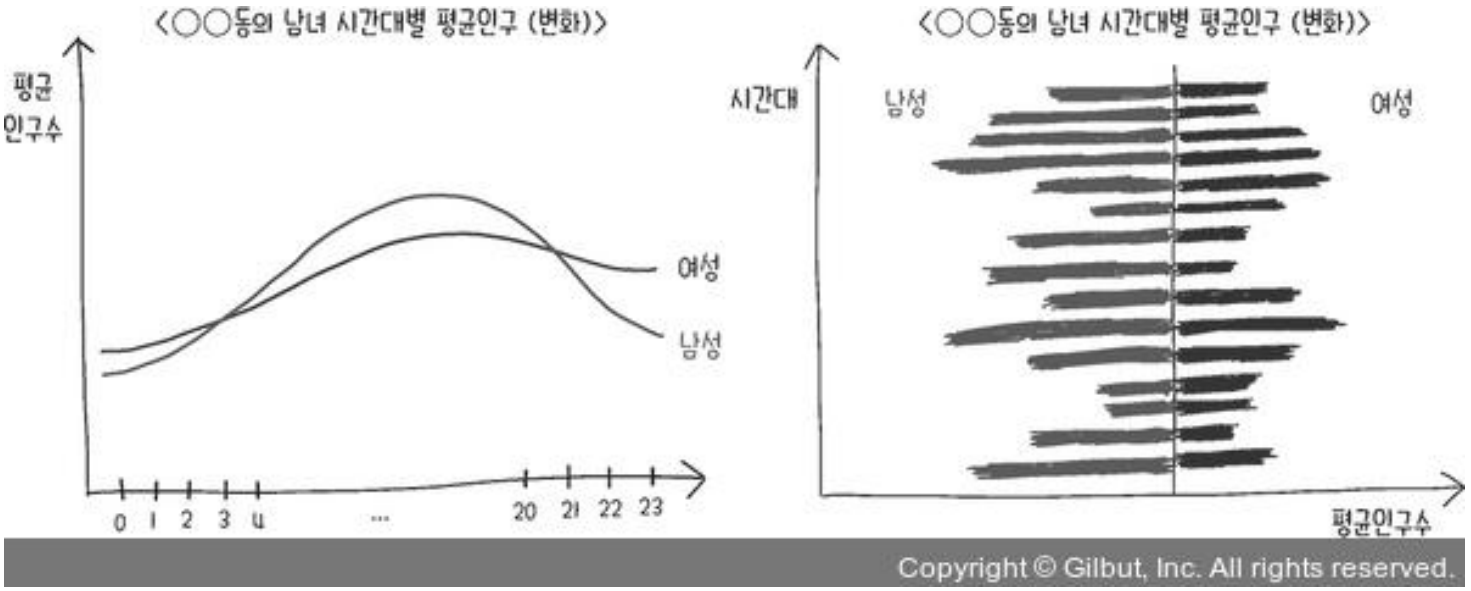
제제쌤의 조언

두 번째 하위 목표를 결정할 때 주중과 주말을 어떻게 구분할지 생각해 보는 과정이 있었습니다. 이처럼 목표 수립 단계에서는 프로그래밍이 가능할지, 가능하다면 어떻게 구현할지 확인하고 하위 목표를 수립하길 추천합니다. 구현 단계에서 시행착오를 줄일 수 있습니다.

더 분석할 거리가 없을까요? 인구 데이터 파일을 보면 다섯 번째 항목부터 남녀의 연령별 생활인구가 나와 있습니다. 이를 사용해 성별 인구도 비교할 수 있습니다. 가려는 곳이 여성이 선호하는 곳인지, 남성이 선호하는 곳인지에 따라 붐비는 시간대가 다를 수 있으니까요. 그럼 세 번째 목표는 핫플레이스가 있는 행정동의 남녀 시간대별 평균인구 그래프를 그려 분석하기로 하고 그래프를 스케치해 봅시다. 이번에는 꺾은선 그래프와 좌우로 펼쳐진 가로형 막대그래프로 표현합니다.

하위 목표 3

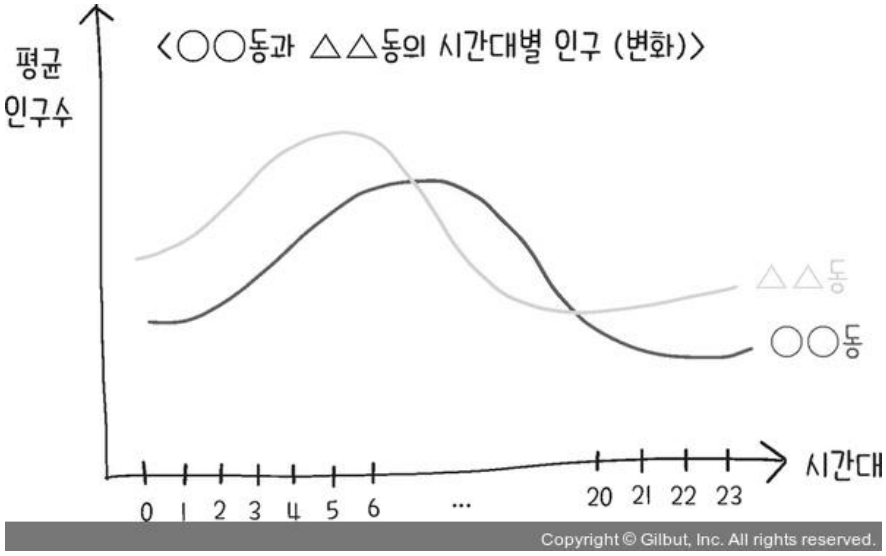
핫플레이스가 있는 행정동의 남녀 시간대별 평균인구 그래프를 그려 분석하기



지금까지 '핫플레이스가 있는 행정동에서 인구가 가장 적은 시간대 파악하기'라는 목표를 달성하는 데 필요한 하위 목표 3가지를 세웠습니다. 핫플레이스가 위치한 행정동의 인구 그래프를 다양한 기준으로 그려 분석하는 것이었죠. 그런데 인구를 한 장소의 절대치로만 보니 실제 어느 정도 붐비는지 잘 와닿지 않을 것 같습니다. 천 명 또는 만 명일 때 어느 정도 붐비는지 실감이 안 나죠. 이럴 때는 자신이 잘 아는 지역과 핫플레이스가 속한 지역을 비교하면 사람이 얼마나 많은지 적은지를 판단할 수 있지 않을까요? 그래서 마지막 하위 목표로 핫플레이스가 있는 행정동과 나에게 익숙한 행정동의 시간대별 평균인구 그래프를 그려 비교 분석하기로 정하고 그래프를 스케치해 봅시다.

하위 목표 4

핫플레이스가 있는 행정동과 나에게 익숙한 행정동의 시간대별 평균인구 그래프를 그려 비교 분석하기



프로젝트 목표를 수립하고 하위 목표까지 모두 작성했습니다. 지금처럼 목표 수립 단계에서 모든 목표를 도출해도 되지만, 실제로는 프로젝트를 진행하면서 새로운 목표가 추가되는 일이 많습니다. 데이터 분석 결과를 보고 나면 이와 연관된 새로운 궁금증이 생기기 때문이죠. 그러니 프로젝트 로드맵을 유연하게 활용하기를 추천합니다!

NOTE 공공 데이터

데이터를 내려받은 서울 열린데이터 광장(https://data.seoul.go.kr)은 누구나 공공 데이터를 이용할 수 있도록 서울시에서 운영하는 사이트입니다. 공공 데이터는 공공 기관이 업무상 작성하거나 취득해 관리하는 자료를 말합니다. 공개된 데이터라면 특별한 인증이나 절차 없이 내려받을 수 있습니다. 인구 데이터는 물론 교통, 교육, 행정 등 여러 분야의 다양한 데이터를 제공합니다. 대표적인 공공 데이터 사이트로 공공데이터포털(https://www.data.go.kr)이 있습니다. 공공데이터포털은 서울시뿐만 아니라 대한민국 정부가 보유한 다양한 공공 데이터를 제공합니다.



▲ 그림 11-12 공공데이터포털

이외에도 구글에서 검색하면 민간 기업이 제공하는 데이터도 구할 수 있습니다.

11.2

프로그램으로 구현하기

두 번째 단계인 프로젝트 구현으로 들어가 보겠습니다. 하위 목표 4가지의 알고리즘을 작성하고 코드로 구현해야죠. 하위 목표를 코드로 구현하기 전에 먼저 해야 할 작업이 있습니다. 내려받은 데이터를 프로그램에서 사용하려면 파이썬 파일에서 읽어 들여야 합니다. 그리고 앞에서 언급했듯이 행정동명에 해당하는 행정동코드를 알아내야 합니다. 그럼 2가지 작업부터 진행해 보겠습니다.

11.2.1 데이터 파일 읽고 행정동명과 행정동코드 연결하기

먼저 인구 데이터인 LOCAL_PEOPLE_DONG_201912.csv와 행정동코드 데이터인 dong_code.csv 파일이 같은 폴더 안에 있는지 확인하세요. 그러고 나서 동일한 폴더에 파이썬 파일을 생성하세요(코랩에서 실행할 경우 파일을 직접 선택할 수 있으므로 같은 폴더가 아니어도 됩니다).

> 제제쌤의 조언
> 생활인구 데이터 파일이 매우 크므로 이 장에서는 구글 코랩이 아닌 주피터 노트북으로 실습하기를 추천합니다. 코랩을 사용하면 파일을 업로드하는 데 시간이 오래 걸립니다. 주피터 노트북은 파일을 업로드하는 방식이 아니어서 파일을 빠르게 읽을 수 있습니다.

데이터 파일 읽어 들이기

9장에서 배운 대로 파이썬 파일에 데이터 파일을 읽어 들여 봅시다. 인구 데이터 파일을 읽어 리스트로 변환한 후 이를 변수(`data`)에 저장합니다. 마지막에 전체 데이터 길이도 출력해 보겠습니다.

```
import csv

f = open('LOCAL_PEOPLE_DONG_201912.csv', encoding='utf8') # 또는 'eur-kr', 'cp949'
data = csv.reader(f)
next(data)
data = list(data)
print(len(data))
```

```
실행결과
315456
```

결과가 출력된다면 데이터 읽기에 성공한 겁니다. 결과를 보니 파일에 약 31만 개의 행이 있네요. 데이터 양이 많으므로 처음 파일을 읽을 때 시간이 좀 걸립니다.

> 제제쌤의 조언
> 인코딩 에러가 발생한다면 utf8 대신 eur-kr 또는 cp949를 넣거나 인코딩 옵션 자체를 삭제해 보세요. 사용하는 PC 환경에 따라 인코딩 옵션이 다르게 적용됩니다.

두 번째 파일인 dong_code.csv도 읽어 봅시다. 앞서 인구 데이터를 읽을 때 `f`와 `data`를 사용했으므로 dong_code.csv 파일을 읽을 때는 다른 변수를 사용해야 데이터가 겹치지 않습니다. 이번에는 `f2`와 `code_data`를 사용하겠습니다.

```
f2 = open('dong_code.csv', encoding = 'utf8')
code_data = csv.reader(f2)
next(code_data)
next(code_data)
code_data = list(code_data)
print(len(code_data))
```

```
실행결과
424
```

dong_code.csv 파일에는 총 424개 행이 있습니다. 행정동 개수가 총 424개라는 의미죠. 그런데 이번에는 `next(code_data)`가 두 줄입니다. dong_code.csv 파일을 보면 헤더가 두 줄이고, 실제 데이터는 세 번째 행부터 있습니다. 그래서 `next(code_data)`를 두 번 실행해 한글 헤더와 영문 헤더를 모두 제거합니다.

	A	B	C	D	E
1	통계청행정동코드	행자부행정동코드	시도명	시군구명	행정동명
2	H_SDNG_CD	H_DNG_CD	DO_NM	CT_NM	H_DNG_NM
3	1101053	11110530	서울	종로구	사직동
4	1101054	11110540	서울	종로구	삼청동
5	1101055	11110550	서울	종로구	부암동
6	1101056	11110560	서울	종로구	평창동
7	1101057	11110570	서울	종로구	무악동

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 11-13 dong_code.csv 파일 헤더

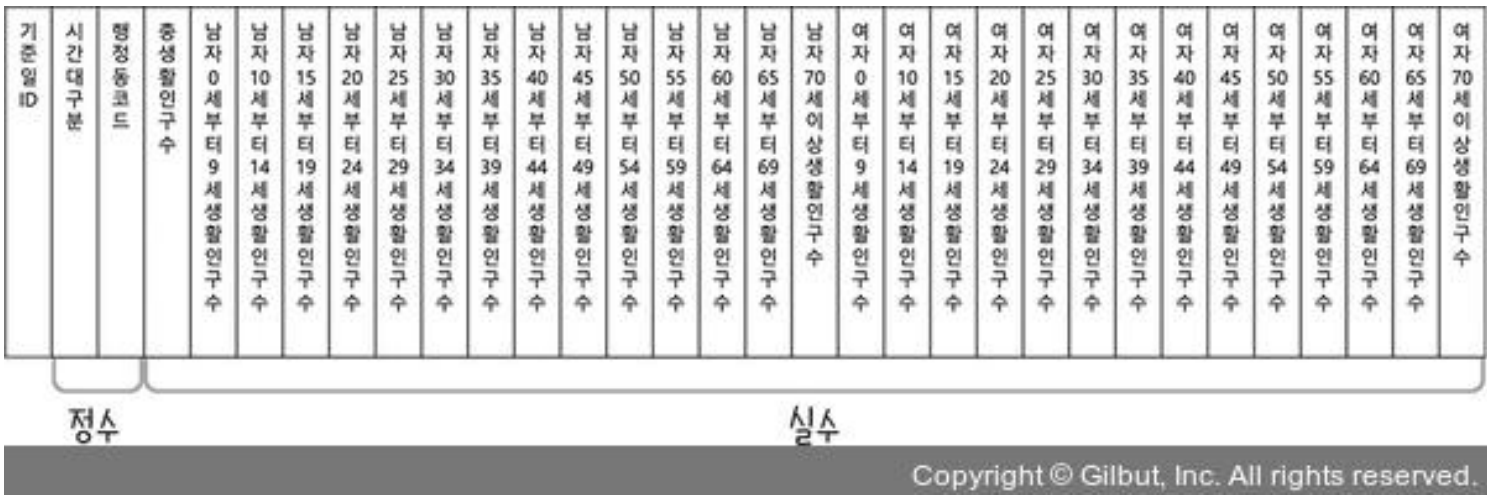
데이터 타입 변환하기

읽어 들인 데이터의 실제 값을 확인해 봅시다. 인구 데이터의 첫 번째 행만 출력해 봅시다.

```
print(data[0])
```

```
실행결과
['20191201', '00', '11215710', '45694.998800000001', '452.3937', '208.7971', '1267.7678000000
001', '5642.9234999999999', '5433.6800999999996', …]
```

데이터가 잘 보입니다. 그런데 숫자가 모두 작은 따옴표로 감싸져 있습니다. CSV 파일을 읽어 들였기 때문에 실제 데이터가 숫자라 할지라도 문자열로 저장돼서 그렇습니다. 우리는 행정동의 평균인구를 구하기 위해 인구수를 더하고 나누는 작업을 해야 합니다. 따라서 데이터는 연산이 가능한 숫자형이어야 하므로 데이터 타입을 변환해야 합니다.



▲ 그림 11-14 서울 생활인구 데이터 파일의 헤더

헤더를 보고 각 항목(열)을 어떻게 변환해야 할지 생각해 봅시다. 먼저 네 번째 열인 총생활인구수부터 마지막 열까지는 모두 소수점이 있는 실수입니다. 따라서 읽어 들인 데이터를 `float()`를 사용해 실수형으로 변환해야 합니다.

그리고 두 번째와 세 번째 열은 모두 정수입니다. 따라서 `int()`를 사용해 정수형으로 변환하면 되죠. 그런데 날짜를 의미하는 첫 번째 열인 기준일ID는 읽어 들인 문자열 그대로 두어야 합니다. 기준일ID는 '20191201' 형태입니다. 날짜는 나중에 슬라이싱해서 연도, 월, 일을 분리합니다. 문자열이 슬라이싱하기가 편하므로 기준일ID만 문자열 그대로 두겠습니다(6장에 나온 문자열 슬라이싱을 참고하세요).

정리하면 시간대구분, 행정동코드는 정수형으로, 총생활인구수부터 마지막 열까지는 실수형으로 변환해야 합니다. 데이터 타입을 변환하는 코드는 다음과 같이 작성할 수 있습니다.

```
for row in data:
    row[1] = int(row[1])        # 시간대구분
    row[2] = int(row[1])        # 행정동코드
    row[3] = float(row[1])      # 총생활인구수
    row[4] = float(row[1])      # 남자0세부터9세생활인구수
    ...
    row[31] = float(row[31])    # 여자70세이상생활인구수
```

`row[1]`부터 `row[31]`까지 모두 바꿔야 하므로 반복문을 사용합니다. 변경해야 할 열이 총 31개이므로 데이터 타입을 31번 변환해야 합니다. 그런데 잘 살펴보면 여기서도 반복되는 부분이 보입니다. 인덱스가 1부터 31까지 1씩 증가하며 데이터 타입을 변환하고 있죠. 이 부분에 반복문을 사용할 수 있습니다. 그리고 인덱스가 2보다 작거나 같을 때는 정수형으로, 2보다 클 때는 실수형으로 변환하면 됩니다.

```
for row in data:
    for i in range(1, 32):
        if i <= 2:      # 인덱스가 2 이하인 경우 정수형으로 변환
            row[i] = int(row[i])
        else:           # 인덱스가 2 초과인 경우 실수형으로 변환
            row[i] = float(row[i])
print(data[0])
```

실행결과

```
['20191201', 0, 11215710, 45694.9988, 452.3937, 208.7971, 1267.7678, 5642.9235, 5433.6801, 24
94.5993, 1691.4831, 1042.7096, 1058.3378, 895.4639, 875.9212, 601.9097, 496.2165, 850.682, 38
8.4005, 180.3721, 1586.7039, 6973.6176, 4259.7022, 2036.1446, 1285.42, 794.6241, 833.7179, 77
2.1417, 773.2152, 827.1455, 565.5378, 1405.3707, '']
```

첫 번째 열인 기준일ID를 제외하고 모든 데이터가 숫자형으로 변경됐습니다. 마지막의 `''`는 존재하지 않는 열인데 파일을 읽어 들이는 과정에서 포함됐습니다. 연산에 활용할 데이터는 아니니 그대로 둬도 괜찮습니다. 인구 데이터를 분석하기 편하도록 데이터 타입을 변환하는 작업을 완료했습니다.

이번에는 행정동코드 데이터를 봅시다.

```
print(code_data[0])
```

```
실행결과
['1101053', '11110530', '서울', '종로구', '사직동']
```

역시 모든 값이 문자열로 저장되어 있습니다. 이 중에서 사용할 항목은 두 번째 열의 행자부 행정동코드와 마지막 열인 행정동명입니다. 행정동명은 문자열 그대로 사용하므로 행자부 행정동코드만 숫자형으로 변경합니다.

```
for row in code_data:
row[1] = int(row[1])
print(code_data[0])
```

```
실행결과
['1101053', 11110530, '서울', '종로구', '사직동']
```

행정동코드 데이터도 데이터 타입 변환이 잘 됐습니다.

행정동명과 행정동코드 연결하기

필요한 데이터를 모두 읽고, 적절한 데이터 타입으로 변환했으니 행정동명과 행정동코드를 연결해 봅시다. 여기서부터는 코드가 복잡하니 먼저 알고리즘을 작성하고 나서 이를 코드로 구현하겠습니다.

분석할 핫플레이스가 위치한 행정동을 선택해야 합니다. 사용자가 알고 싶은 행정동명을 직접 입력하게 해 봅시다. 사용자가 서울시의 행정동 중 하나를 입력하면 행정동명과 행정동코드의 매핑 정보가 담긴 dong_code.csv에서 해당하는 행정동코드를 찾습니다.

알고리즘

```
1. 사용자에게서 행정동명을 입력받아 변수(dong_name)에 저장하기
2. 행정동코드 데이터(code_data)를 돌며 반복하기
    2.1 행정동코드 데이터의 마지막 열인 행정동명(열 인덱스[-1])이 입력된 행정동명(dong_name)과 같다면
        2.1.1 해당하는 행정동코드를 변수(dong_code)에 저장하기
```

과정 1은 간단하게 행정동명을 `input()`으로 입력받아 변수 `dong_name`에 저장하면 됩니다. 과정 2에서 행정동코드를 구하려면 dong_code.csv 파일의 데이터를 담고 있는 `code_data`를 사용해야 합니다. `code_data`에서 행정동명은 마지막 열에 있죠(dong_code.csv 파일을 확인해 보세요). `code_data` 전체를 돌면서 입력받은 행정동명(`dong_name`)이 `code_data`의 마지막 열(열 인덱스 [-1])과 같을 때 해당 행의 행정동코드를 `dong_code`에 저장합니다.

알고리즘에 해당하는 코드를 작성해 봅시다(지면상 코드 부분은 들여쓰기를 생략합니다).

> 〈알고리즘〉
>
> 1. 사용자에게서 행정동명을 입력받아 변수(dong_name)에 저장하기
> 2. 행정동코드 데이터(code_data)를 돌며 반복하기
> 2.1 행정동코드 데이터의 마지막 열인 행정동명(열 인덱스 [-1])이 입력된 행정동명(dong_name)과 같다면
> 2.1.1 해당하는 행정동코드를 변수(dong_code)에 저장하기
>
> ---
>
> 〈코드〉

```
1. dong_name = input()
2. for row in code_data:
    2.1 if row[-1] == dong_name:
        2.1.1 dong_code = row[1]
```

코드를 프로그램으로 옮겨 봅시다. 알아보기 쉽게 안내문도 적당히 추가합니다.

```
dong_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
for row in code_data:
    if row[-1] == dong_name:
        dong_code = row[1]
print(dong_name, '-', dong_code, '을(를) 분석합니다!')
```

프로그램을 실행해 압구정동을 입력해 봅시다.

```
실행결과
핫플레이스가 위치한 행정동을 입력하세요. --> 압구정동
압구정동 - 11680545 을(를) 분석합니다!
```

입력된 행정동명(압구정동)과 행정동코드(11680545)가 각각 `dong_name`, `dong_code`에 저장됐음을 확인할 수 있습니다. 이후 구현 과정에서 `dong_name`과 `dong_code` 변수를 활용합니다.

이 책에서는 압구정동을 기준으로 프로젝트를 진행합니다. 다른 지역이 궁금하다면 바꿔서 입력해 보고 결과가 어떻게 달라지는지 직접 확인해 보세요.

핫플레이스 인구 분석 프로젝트에서 실행해야 하는 기본 코드를 정리하면 다음과 같습니다.

```
import csv

# 데이터 파일 읽어 들이기
f = open('LOCAL_PEOPLE_DONG_201912.csv', encoding='utf8') # 또는 'eur-kr', 'cp949'
data = csv.reader(f)
next(data)
data = list(data)

f2 = open('dong_code.csv', encoding = 'utf8') # 또는 'eur-kr', 'cp949'
code_data = csv.reader(f2)
next(code_data)
next(code_data)
code_data = list(code_data)

# 데이터 타입 변환하기
# 인구 데이터 변환하기
for row in data:
    for i in range(1, 32):
        if i <= 2:      # 인덱스가 2 이하인 경우 정수형으로 변환
            row[i] = int(row[i])
        else:           # 인덱스가 2 초과인 경우 실수형으로 변환
            row[i] = float(row[i])

# 행정동코드 변환하기
for row in code_data:
    row[1] = int(row[1])

# 행정동명과 행정동코드 연결하기
dong_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
for row in code_data:
    if row[-1] == dong_name:
        dong_code = row[1]
print(dong_name, '-',dong_code, '을(를) 분석합니다!')
```

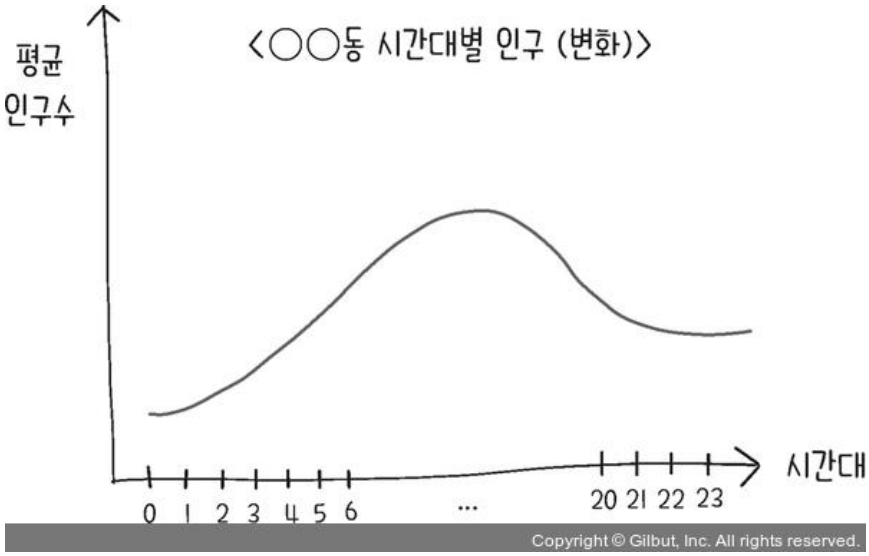
이제 프로젝트 목표 수립 단계에서 정리한 하위 목표를 하나씩 프로그램으로 구현해 봅시다.

11.2.2 하위 목표 1 - 시간대별 인구 분석하기

첫 번째 하위 목표는 '핫플레이스가 있는 행정동의 시간대별 평균인구 그래프를 그려 분석하기'입니다. 알고리즘부터 작성해 봅시다.

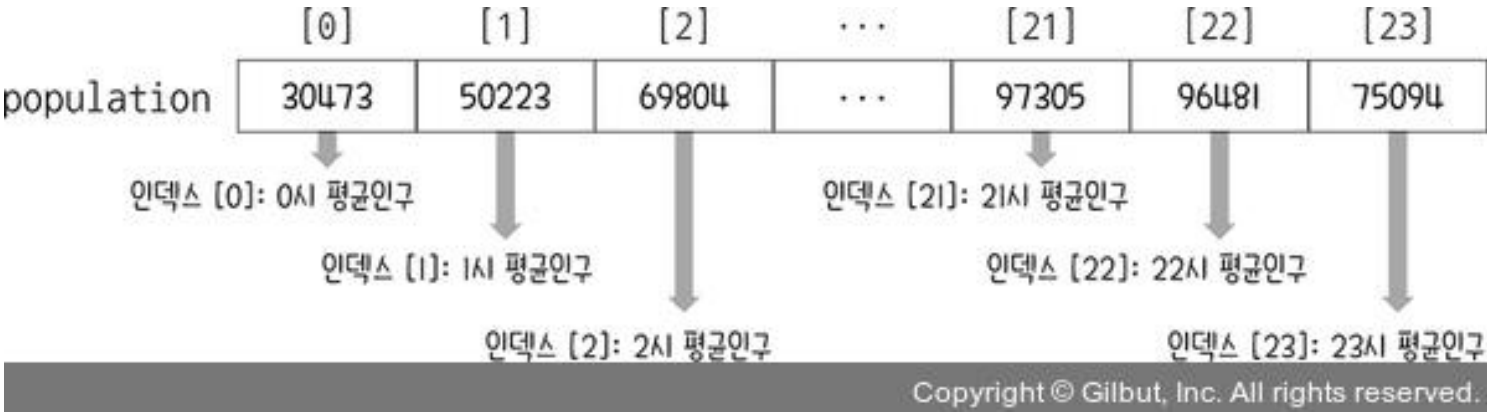
하위 목표 1

핫플레이스가 있는 행정동의 시간대별 평균인구 그래프를 그려 분석하기



알고리즘 작성하기

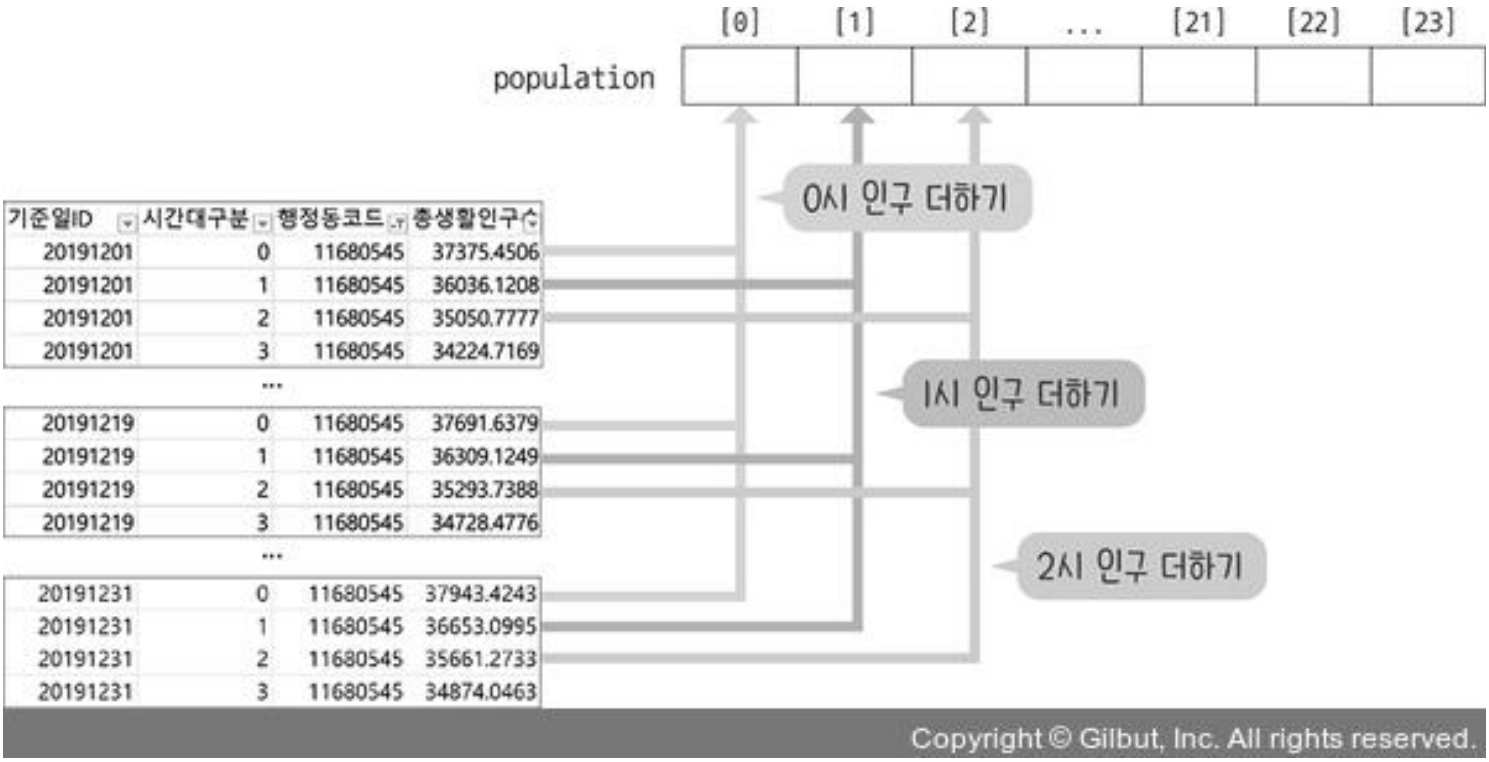
하위 목표 1에서는 x축을 시간대, y축을 시간대별 평균인구로 하는 꺾은선 그래프를 그려야 합니다. 꺾은선 그래프는 `plt.plot()`으로 그립니다. `plt.plot()`에는 x축 리스트와 y축 리스트를 각각 넣어야 하죠. x축은 0~23이라는 연속된 숫자 범위를 나타내므로 `range(24)`로 작성할 수 있습니다. y축에 넣는 리스트는 행정동의 시간대별 평균인구를 담고 있어야 합니다. 인구를 저장하니 리스트명은 `population`으로 합시다. `population` 리스트의 구조는 다음과 같습니다.



▲ 그림 11-15 population 리스트의 구조

y축 리스트는 x축과 개수가 같아야 하므로 24개 요소가 있는 일차원 리스트입니다. `population`은 시간대별 평균인구를 담고 있으므로 0시 평균인구는 `population[0]`에, 1시 평균인구는 `population[1]`에 각각 저장하는 방식으로 `population[23]`까지 저장합니다. 그림을 보면 `population` 리스트의 인덱스는 시간대를 의미합니다.

리스트 구조를 알았으니 데이터를 채우면 됩니다. 리스트에 데이터는 어떻게 채울까요? 현재 `dong_name`과 `dong_code`에 저장된 '압구정'과 '11680545'를 기준으로 생각해 봅시다. `data` 리스트에는 2019년 12월 1일부터 2019년 12월 31일까지 총 31일간의 시간대별 인구 데이터가 들어 있습니다. 이 중에서 압구정동 인구 데이터만 고르면 31일 × 24시 = 744개 행입니다. LOCAL_PEOPLE_DONG_201912.csv 파일에서 압구정동 부분만 잘라서 보면 다음과 같습니다. 이 데이터가 `data` 리스트에서 행정동코드가 '11680545'인 행에 들어갑니다.



▲ 그림 11-16 population 리스트에 인구 데이터 저장하기

744개 행에서 같은 시간대의 인구를 더하면 시간대별 총생활인구수를 구할 수 있습니다. 예를 들어, 0시의 총생활인구라면 각 날짜의 0시 총생활인구(12월 1일 0시 총생활인구 + 12월 2일 0시 총생활인구 + … + 12월 31일 0시 총생활인구)를 모두 더해 구합니다. 그리고 이 값을 `population[0]`에 저장하면 됩니다. 같은 방법으로 `population[1]`에는 12월 1일부터 31일까지의 1시 총생활인구 합을 구해 넣습니다.

정리하면 `data`의 행정동코드(열 인덱스 [2])가 `dong_code`의 값과 같은 행을 찾고 해당 행의 시간대구분과 총생활인구수를 구합니다. 시간대구분은 `population`의 인덱스이므로 시간대구분에 해당하는 총생활인구수를 `population[시간대]`에 더하면 됩니다. 이 과정을 `data`의 첫 행부터 마지막 행까지 반복합니다. `population`의 각 요소에는 `dong_code` 시간대별 총생활인구수의 합이 저장됩니다. 이를 평균인구로 바꾸기 위해 `population` 리스트의 모든 요소를 31로 나눈 후 다시 저장합니다. 이 과정을 거치면 입력한 행정동(`dong_code`)의 시간대별 평균인구가 `population` 리스트에 차례대로 담기게 됩니다. 이 과정을 알고리즘으로 작성해 봅시다.

하위 목표 1

> 핫플레이스가 있는 행정동의 시간대별 평균인구 그래프를 그려 분석하기

알고리즘

> 1. 시간대별 평균인구를 저장할 리스트(population)를 길이 24로 만들고 초깃값 0 저장하기
> 2. 인구 데이터(data)의 첫 행부터 마지막 행까지 반복하기
> 2.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드 (열 인덱스 [2])가 같다면
> 2.1.1 해당 행의 시간대(열 인덱스 [1])와 총생활인구수(열 인덱스 [3])를 각각 변수(time, p)에 저장하기
> 2.1.2 과정 2.1.1에서 저장한 시간대(time)가 population의 인덱스이므로 population[time]에 총생활인구수(p) 더하기
> 3. 반복이 끝나면 population 리스트의 모든 값을 31로 나눈 후 population에 다시 저장하기
> 4. 완성된 population 리스트로 시간대별 평균인구 그래프 그리기

코드 작성하기

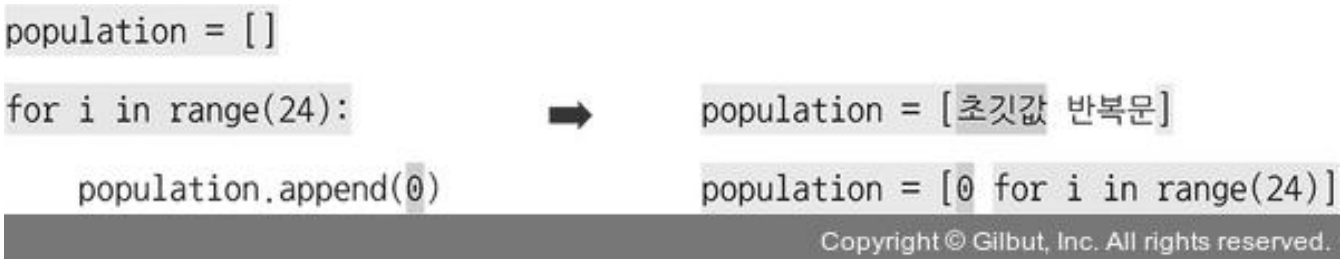
작성한 알고리즘대로 프로그래밍해 봅시다. 카드 이용내역 분석에서 한 것처럼 알고리즘 옆에 단계별 코드를 작성하면 편합니다.

과정 1부터 작성해 봅시다. 시간대별 평균인구를 저장할 리스트 `population`을 만듭니다. 배운 대로 구현한다면 다음처럼 빈 리스트를 만들고 0을 추가하는 작업을 24번 반복할 겁니다.

```
population = []
for i in range(24):
    population.append(0)
```

이렇게 작성하면 코드는 세 줄입니다. 그런데 이 코드를 한 줄로 줄일 방법이 있습니다. 리스트를 처음 배울 때는 어려울 수 있어서 언급하지 않았지만, 실제 프로그램을 작성할 때는 알아 두면 유용하므로 여기서 설명하겠습니다.

반복문을 리스트 안에 넣어 리스트를 만드는 동시에 초기화하는 방법입니다.



Copyright © Gilbut, Inc. All rights reserved.

코드는 매우 비슷합니다. 빈 리스트를 만들 때처럼 리스트를 선언하고 대괄호(`[]`) 안에 초깃값과 반복문을 넣어 줍니다. 초깃값은 0이고 24번 반복하므로 `population = [0 for i in range(24)]`가 됩니다. 이때 반복문에 콜론(:)은 넣지 않습니다. 구조를 잘 살펴보면 `population.append()`가 빠지고 나머지 코드가 대괄호 안에 들어갑니다. 이처럼 리스트 안에 반복문을 포함하는 것을 리스트 내포라고 합니다.

```
population = [0 for i in range(24)]
print(len(population), population)
```

```
실행결과
24 [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

길이가 24인 `population` 리스트가 생성됐고, 각 요소에도 초깃값 0이 저장됐습니다.

이를 조금만 변형해서 `population`에 숫자 0부터 23까지 차례대로 넣는 방법을 생각해 봅시다. 반복문 `for i in range(24)`를 실행하면 `i`에 0부터 23까지 차례대로 담기죠. 따라서 앞의 초깃값 0 대신 `i`를 넣어 `population = [i for i in range(24)]`라고 작성하면 됩니다. 실행해 봅시다.

```
population = [i for i in range(24)]
print(len(population), population)
```

```
실행결과
24 [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]
```

이처럼 리스트 생성과 동시에 초기화하는 방법을 활용하면 코드가 더욱 간결해집니다.

과정 1의 설명이 길어졌네요. 마무리하고 나머지 코드도 작성해 봅니다. 스스로 코드를 완성해 보고 싶으면 표의 오른쪽 부분은 가리고 코드를 작성해 보세요.

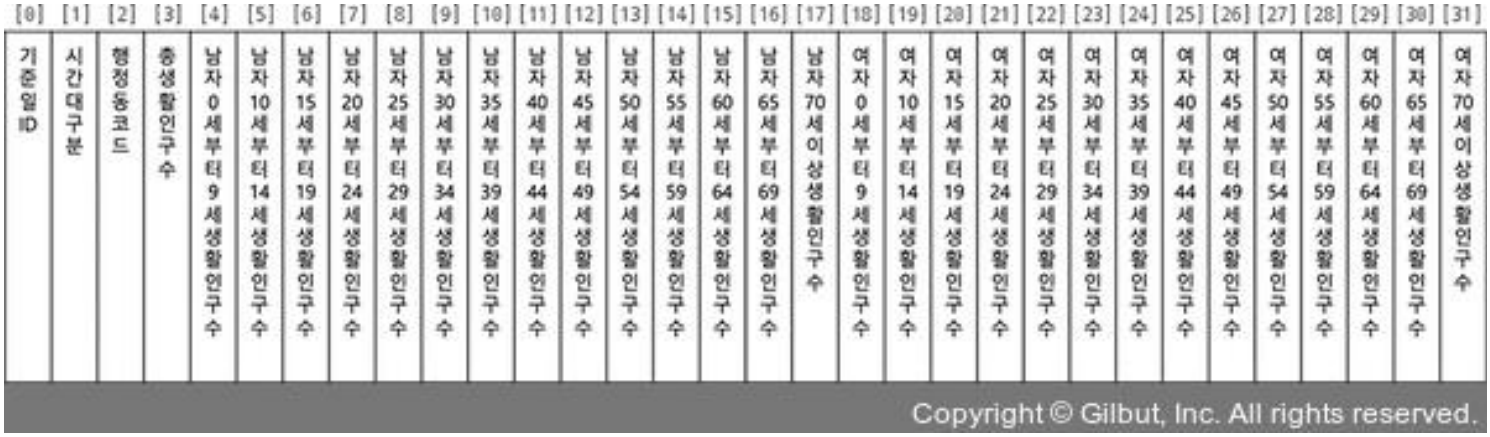
〈알고리즘〉

1. 시간대별 평균인구를 저장할 리스트(population)를 길이 24로 만들고 초깃값 0 저장하기
2. 인구 데이터(data)의 첫 행부터 마지막 행까지 돌며 반복하기
   2.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
      2.1.1 해당 행의 시간대(열 인덱스 [1])와 총생활인구수(열 인덱스 [3])를 각각 변수(time, p)에 저장하기
      2.1.2 과정 2.1.1에서 저장한 시간대(time)가 population의 인덱스이므로 population[time]에 총생활인구수(p) 더하기
3. 반복이 끝나면 population 리스트의 모든 값을 31로 나눈 후 population에 다시 저장하기
4. 완성된 population 리스트로 시간대별 평균인구 그래프 그리기

〈코드〉

```
1. population = [0 for i in range(24)]
2. for row in data:
    2.1 if row[2] == dong_code:
        2.1.1 time, p = row[1], row[3]
        2.1.2 population[time] += p
3. population = [p/31 for p in population]
4. plt.plot(range(24), population)
```

카드 이용내역을 분석할 때도 같은 형식으로 했기 때문에 익숙할 테니 데이터의 차이만 짚고 넘어가겠습니다. `data`에는 총 32개 항목(열)이 있습니다. 데이터 항목 위에 열 인덱스를 넣어보면 다음과 같습니다. 실제로 `data`에는 헤더가 없지만, 각 열이 의미하는 바를 파악하기 쉽게 데이터 파일의 헤더를 보면서 진행하겠습니다.



▲ 그림 11-17 인구 데이터의 헤더와 인덱스

과정 1에서 만든 `population` 리스트에 시간대별 평균인구를 담는 과정을 인구 데이터(`data`)의 첫 행부터 마지막 행까지 반복해야 하므로 과정 2에서는 이를 수행할 반복문을 작성합니다. 반복문에서 `data`의 한 행씩 `row` 변수에 담깁니다.

과정 2.1에서는 사용자가 입력한 행정동의 행정동코드(`dong_code`)와 `data`의 행정동코드가 같은지 한 행씩 확인합니다. `data`에서 행정동코드는 열 인덱스 [2]에 있으므로 `row[2]`가 됩니다. 따라서 `dong_code`와 `row[2]`가 같은지 `if` 조건문으로 확인합니다. 일치하는 행을 찾으면 과정 2.1.1로 가고 조건을 충족하지 않으면 다음 반복 회차로 넘어갑니다.

과정 2.1.1에서는 뒤에서 계산하는 데 필요한 시간대와 총생활인구수를 각각 변수 `time`과 `p`에 저장합니다. 이때 시간대는 열 인덱스 [1]에, 총생활인구수는 열 인덱스 [3]에 있으므로 `time, p = row[1], row[3]`으로 한 번에 저장합니다. 과정 2.1.2에서는 시간대(`time`)를 인덱스로 하는 `population` 리스트의 요소에 총생활인구수(`p`)를 더합니다.

그럼 과정 2 전체를 코드로 작성한 후 `population` 값을 확인해 봅시다.

> TIP 이 코드를 작성할 때는 앞서 작성한 파일 읽기와 행정동명 검색하기가 이미 실행되어야 합니다. 즉, `dong_code`에 11680545가 저장된 상태여야 오류가 발생하지 않습니다.

```
population = [0 for i in range(24)]
for row in data:
    if row[2] == dong_code:
        time, p = row[1], row[3]
        population[time] += p
print(population)
```

```
실행결과
[1150057.7245, 1112646.2784000002 … 1350683.4461, 1220104.0514]
```

압구정동의 시간대별 총생활인구수의 합이 나옵니다. 이 숫자는 12월 한 달간의 시간대별 총생활인구이니 이제 하루 평균인구를 구해야 합니다.

NOTE 코드 줄이기

반복문의 코드 줄 수를 줄이고 싶다면 변수 `time`과 `p`를 사용하지 않아도 됩니다. `time`은 곧 `row[1]`이고, `p`는 `row[3]`이므로 이를 대체해 작성해도 결과는 같습니다.

```
for row in data:
    if row[2] == dong_code:
        population[row[1]] += row[3]
```

반복문이 끝나면 과정 3에서 `population` 리스트를 31로 나누어 시간대별 평균인구를 구합니다. 이때도 리스트 내포를 활용합니다. `population`의 각 요소를 `p`에 차례대로 담은 후 `p`를 31로 나눈 값을 리스트에 다시 저장하면 됩니다.

```
population = [p/31 for p in population]
print(population)
```

실행결과

```
[37098.63627419355, 35891.815432258074 … 43570.433745161296, 39358.195206451615]
```

`population`의 모든 요소를 31로 나누니 숫자가 확 줄었습니다. 0시 인구는 3만7천 명, 1시 인구는 3만6천 명 정도입니다. 오후에 사람이 많을 때는 대략 6만3천 명까지 있고요.

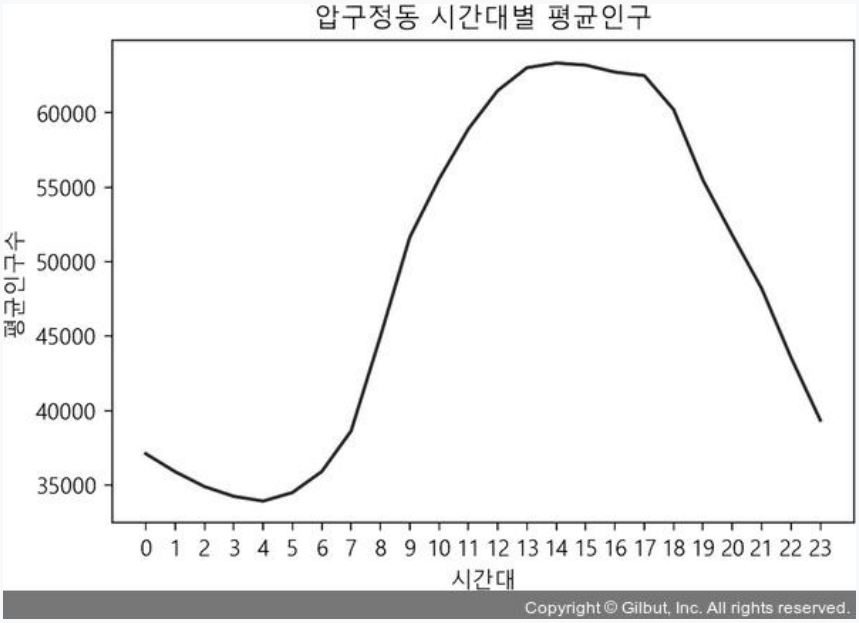
이제 과정 4의 그래프만 그리면 됩니다. 그래프를 그릴 때 필요한 라이브러리인 `matplotlib`을 추가해 꺾은선 그래프를 그려 봅시다. 제목과 x축, y축 이름도 적절히 추가해 보세요. 완성된 코드는 다음과 같습니다.

```
import matplotlib.pyplot as plt

population = [0 for i in range(24)]
for row in data:
    if row[2] == dong_code:
        time, p = row[1], row[3]
        population[time] += p
population = [p/31 for p in population]

plt.rc('font', family='Malgun Gothic')           # macOS에서는 AppleGothic
plt.title(dong_name + ' 시간대별 평균인구')      # 그래프 제목
plt.plot(range(24), population, color='indigo') # 그래프 그리기
plt.xticks(range(24), range(24))                 # x축 눈금
plt.xlabel('시간대')                             # x축 이름
plt.ylabel('평균인구수')                         # y축 이름
plt.show()
```

실행결과



`indigo` 색상의 꺾은선 그래프가 그려집니다. `plt.xlabel()`, `plt.ylabel()`로 x축과 y축 이름도 붙였습니다. `plt.title()`에서 `dong_name` 변수를 넣어 입력한 행정동에 맞춰 그래프 제목이 출력되도록 했습니다. 그리고 `plt.xticks(range(24), range(24))`를 추가해 x축을 1시간 단위로 표기했습니다(넣지 않으면 눈금이 5시간 단위로 표시됩니다).

결과 분석하기

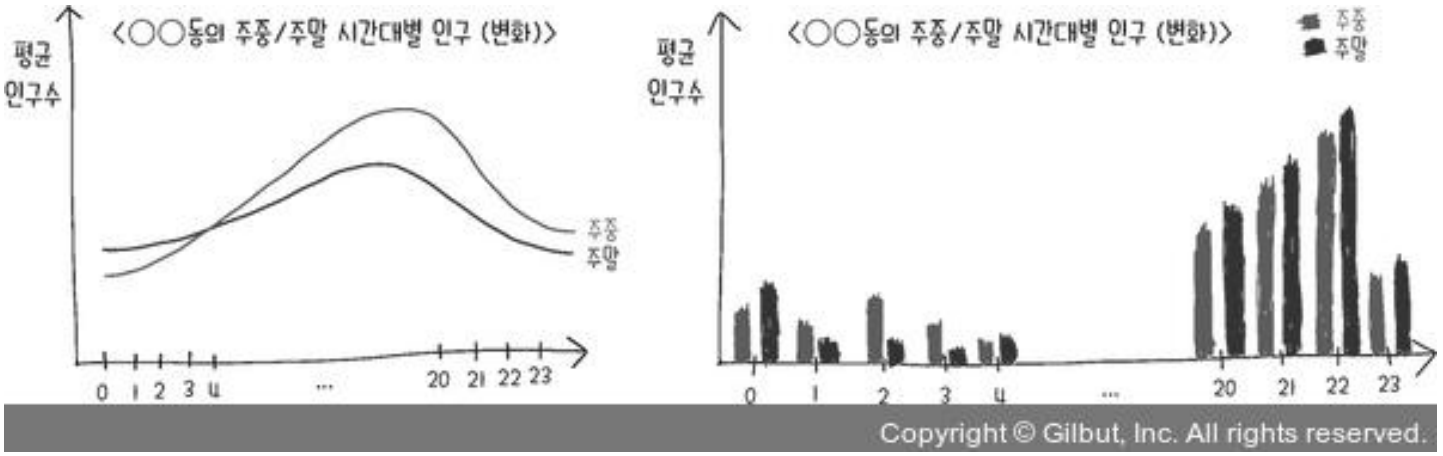
하위 목표 1의 결과를 보니 압구정동에서 인구가 가장 적을 때는 1시에서 6시 사이입니다. 새벽에는 인구 움직임이 매우 적으니 새벽 생활인구 대부분은 압구정에 거주하는 사람일 것 같네요. 8시부터 인구가 증가합니다. 직장인들이 출근하거나 학생들이 등교하나 봅니다. 가장 인구가 많은 시간대는 12시부터 18시까지입니다. 이 시간대에는 평균인구가 약 6만 명이므로 이 시간에 압구정을 방문하면 평소보다 붐비겠네요. 압구정에서는 12~18시를 피해 오전 10~11시쯤 브런치 약속이나 19시 이후에 저녁 약속을 잡으면 좋을 것 같습니다.

11.2.3 하위 목표 2 - 주중/주말 시간대별 인구 분석하기

이번에는 두 번째 하위 목표인 '핫플레이스가 있는 행정동의 주중/주말 시간대별 평균인구 그래프를 그려 분석하기'를 구현해 보겠습니다.

하위 목표 2

핫플레이스가 있는 행정동의 주중/주말 시간대별 평균인구 그래프를 그려 분석하기



주중과 주말 2개의 그래프를 그려야 하므로 주중 인구와 주말 인구가 담긴 리스트가 필요합니다. 목표 수립 단계에서 주중과 주말 구분은 `datetime` 라이브러리를 사용한다고 했죠. 정확히는 `datetime` 라이브러리의 `date().weekday()` 명령어를 사용합니다. `date()` 안에 연도, 월, 일을 순서대로 넣은 후 `weekday()`를 붙이면 월, 화, 수, 목, 금, 토, 일을 각각 숫자 0, 1, 2, 3, 4, 5, 6으로 나타냅니다. 따라서 실행했을 때 0부터 4 사이의 숫자가 나오면 주중이고 5와 6이 나오면 주말입니다. 이를 활용해 주중 인구 리스트와 주말 인구 리스트를 만들 수 있습니다.

2019년 12월 15일의 요일을 확인해 봅시다.

```
import datetime

# 0-월, 1-화, 2-수, 3-목, 4-금, 5-토, 6-일
datetime.date(2019, 12, 15).weekday()
```

```
실행결과
6
```

6이 나오므로 일요일이네요. 달력을 확인해 보면 2019년 12월 15일은 일요일이 맞습니다. 오늘 날짜를 넣어서 실행해 보고 요일에 대응하는 숫자가 올바르게 출력되는지 확인해 보세요.

알고리즘 작성하기
그럼 알고리즘을 작성해 봅시다. 일단 큰 틀은 평균인구를 구할 때와 비슷하지만, 다른 점도 있습니다. 평균인구는 리스트 하나를 사용했지만, 이번에는 그래프 2개를 겹쳐 그리기 때문에 주중 인구와 주말 인구를 저장한 2개의 리스트가 필요합니다. 그리고 반복문으로 시간대별 총생활인구수를 구할 때 주중과 주말을 판단해 주중이면 주중 인구 리스트에, 주말이면 주말 인구 리스트에 더해야 합니다. 주중 인구의 평균과 주말 인구의 평균도 각각 구해야죠. 주중이 주말보다 날짜가 많으니 평균 작업을 제대로 해 주지 않으면 심한 데이터 왜곡이 생길 수 있습니다. 따라서 2019년 12월의 주중 일수와 주말 일수를 구한 후, 이를 활용해 주중과 주말 인구의 평균을 구해야 합니다.

하위 목표 2

> 핫플레이스가 있는 행정동의 주중/주말 시간대별 평균인구 그래프를 그려 분석하기

알고리즘

> 1. 주중 시간대별 생활인구를 저장할 리스트(weekday)와 주말 시간대별 생활인구를 저장할 리스트(weekend)를 길이 24로 만들고 초깃값 0 저장하기
> 2. 인구 데이터(data)의 첫 행부터 마지막 행까지 돌며 반복하기
> 2.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
> 2.1.1 해당 행의 시간대(time)와 총생활인구수(p), 기준일ID의 연도(year), 월(mon), 일(day)을 각각 변수에 저장하기
> 2.1.2 과정 2.1.1에서 저장한 연도(year), 월(mon), 일(day)을 datetime.date().weekday()에 넣어 주중/주말 구분하기
> 2.1.3 주중이면 주중 리스트(weekday[time])에 총생활인구수(p), 주말이면 주말 리스트(weekend[time])에 총생활인구수(p) 더하기
> 3. 반복이 끝나면 2019년 12월의 주중 일수(weekday_cnt)와 주말 일수(weekend_cnt)를 저장할 변수를 만들고 초깃값 0 저장하기
> 4. 2019년 12월의 일수만큼 반복하기
> 4.1 주중이면 주중 일수(weekday_cnt)에 1, 주말이면 주말 일수(weekend_cnt)에 1 더하기
> 5. 주중 리스트(weekday)의 각 요소를 주중 일수(weekday_cnt)로, 주말 리스트(weekend)의 각 요소를 주말 일수(weekend_cnt)로 나누어 주중/주말 평균인구 구하기
> 6. 완성된 weekday와 weekend 리스트로 주중/주말 시간대별 평균인구 그래프 그리기

하위 목표 1의 알고리즘과 비교했을 때 과정 2가 가장 크게 변했네요. 과정 2.1.1에서 구한 연도, 월, 일을 활용해 각 행이 주중인지 주말인지를 판단하고, 주중이면 주중 리스트, 주말이면 주말 리스트에 총생활인구수를 더합니다. 과정 3에서는 2019년 12월의 주중 일수와 주말 일수를 반복문으로 구합니다. 그리고 과정 4에서 오차가 생기지 않게 주중 일수와 주말 일수로 정확히 나눕니다.

코드 작성하기

알고리즘을 바탕으로 코드를 작성해 봅시다. 하위 목표 1의 알고리즘과 비슷하기 때문에 알고리즘을 코드로 옮기기는 어렵지 않습니다. 다음 표의 오른쪽 부분을 가리고 직접 작성해 본 후 책과 비교해 보세요. 참고로 2019년 12월의 주중은 총 22일, 주말은 총 9일입니다.

> 〈알고리즘〉
>
> 1. 주중 시간대별 생활인구를 저장할 리스트(weekday)와 주말 시간대별 생활인구를 저장할 리스트(weekend)를 길이 24로 만들고 초깃값 0 저장하기
> 2. 인구 데이터(data)의 첫 행부터 마지막 행까지 반복하기
>   2.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
>   2.1.1 해당 행의 시간대(time)와 총생활인구수(p), 기준일ID의 연도(year), 월(mon), 일(day)을 각각 변수에 저장하기
>   2.1.2 과정 2.1.1에서 저장한 연도(year), 월(mon), 일(day)을 datetime.date().weekday()에 넣어 주중/주말 구분하기
>   2.1.3 주중이면 주중 리스트(weekday[time])에 총생활인구수(p), 주말이면 주말 리스트(weekend[time])에 총생활인구수(p) 더하기
> 3. 반복이 끝나면 2019년 12월의 주중 일수(weekday_cnt)와 주말 일수(weekend_cnt)를 저장할 변수를 만들고 초깃값 0 저장하기
> 4. 2019년 12월의 일수만큼 반복하기
>   4.1 주중이면 주중 일수(weekday_cnt)에 1, 주말이면 주말 일수(weekend_cnt)에 1 더하기
> 5. 주중 리스트(weekday)의 각 요소를 주중 일수(weekday_cnt)로, 주말 리스트(weekend)의 각 요소를 주말 일수(weekend_cnt)로 나누어 주중/주말 평균인구 구하기
> 6. 완성된 weekday와 weekend 리스트로 주중/주말 시간대별 평균인구 그래프 그리기
>
> ---

〈코드〉

```
1. weekday = [0 for i in range(24)]
   weekend = [0 for i in range(24)]
2. for row in data:
    2.1 if row[2] == dong_code:
        2.1.1 time, p = row[1], row[3]
              year, mon, day = int(row[0][:4]), int(row[0][4:6]), int(row[0][6:])
        2.1.2 num = datetime.date(year, mon, day).weekday()
        2.1.3 if num < 5:
                  weekday[time] += p
              else:
                  weekend[time] += p
3. weekday_cnt, weekend_cnt = 0,0
4. for i in range(1, 32):
    4.1 if datetime.date(2019, 12, i).weekday() < 5:
            weekday_cnt += 1
        else:
            weekend_cnt += 1
5. weekday = [w/weekday_cnt for w in weekday]
   weekend = [w/weekend_cnt for w in weekend]
6. plt.plot(range(24), weekday)
   plt.plot(range(24), weekend)
```

과정 2.1까지는 하위 목표 1과 리스트명만 다르고 거의 비슷하므로 바로 과정 2.1.1을 봅시다. 기준일ID는 `data`의 열 인덱스 [0] `(row[0])`에 있습니다. 뒤에서 주중과 주말을 구분하는 작업할 때 연도, 월, 일이 각각 필요합니다. 그런데 날짜가 '20191201'처럼 공백 없이 붙어 있으므로 이를 분리하는 작업이 필요합니다. 6장에서 배운 문자열 슬라이싱을 적용하면 됩니다. `row[0]`에 들어 있는 여덟 글자 중 연도를 의미하는 앞 네 자리만 잘라내려면 `row[0][:4]`로 작성합니다. 잘라낸 값은 데이터 타입이 문자열이므로 `int()`로 감싸 정수형으로 변경한 후 변수 `year`에 저장합니다. 같은 방법으로 월(`mon`)과 일(`day`)을 저장합니다.

과정 2.1.2에서는 과정 2.1.1에서 구한 연도(`year`), 월(`mon`), 일(`day`)을 `datetime.date().weekday()`에 넣어 결과로 나온 값을 변수 `num`에 저장합니다. `num`이 4 이하면 주중, 5 또는 6이면 주말이므로 과정 2.1.3에서 `if-else` 문으로 주중, 주말을 판단합니다. 판단 결과 주중이면 `weekday` 리스트의 같은 시간대(`[time]`)에, 주말이면 `weekend` 리스트의 같은 시간대(`[time]`)에 총생활인구수 `p`를 더하면 됩니다.

과정 4에서는 주중과 주말 일수를 구합니다. 연도와 월은 2019년 12월로 같지만, 일은 1일부터 31일까지 바뀌기 때문에 반복문을 사용합니다. `datetime.date().weekday(2019, 12, 일)`이 5 미만이면 주중이므로 주중 일수를 의미하는 `weekday_cnt`에 1을 더하고, 아니라면 주말이므로 주말 일수인 `weekend_cnt`에 1을 더합니다.

과정 5에서는 주중과 주말 일수를 활용해 주중과 주말의 평균인구를 구합니다. `weekday` 리스트의 모든 요소를 `weekday_cnt`로, `weekend` 리스트의 모든 요소를 `weekend_cnt`로 나누면 됩니다.

그럼 처음부터 과정 5까지 프로그램으로 옮겨 봅시다. 필요한 라이브러리를 추가하고 알고리즘 옆의 코드를 차례대로 적습니다. 리스트 `weekday`와 `weekend`에 값이 제대로 들어갔는지 확인하기 위해 마지막에 `print()`를 추가합니다.

```
import datetime

weekday = [0 for i in range(24)]
weekend = [0 for i in range(24)]

for row in data:
    if row[2] == dong_code:
        time, p = row[1], row[3]
        year, mon, day = int(row[0][:4]), int(row[0][4:6]), int(row[0][6:])
        num = datetime.date(year, mon, day).weekday()
        if num < 5:
            weekday[time] += p
        else:
            weekend[time] += p

# 2019년 12월의 주중/주말 일수 구하기
weekday_cnt, weekend_cnt = 0, 0
for i in range(1, 32):
    if datetime.date(2019, 12, i).weekday() < 5:
        weekday_cnt += 1
    else:
        weekend_cnt += 1

print('2019년 12월의 주중 일수 =', weekday_cnt, '주말 일수 =',weekend_cnt)

weekday = [w/weekday_cnt for w in weekday]
weekend = [w/weekend_cnt for w in weekend]

print('주중 인구:', weekday)
print('주말 인구:', weekend)
```

```
실행결과
2019년 12월의 주중 일수 = 22 주말 일수 = 9
주중 인구: [36348.0879, 35284.529854545464… 45000.37916363635, 40292.146422727266]
주말 인구: [38933.31007777777, 37376.29128888888… 40075.01161111111, 37075.203344444446]
```

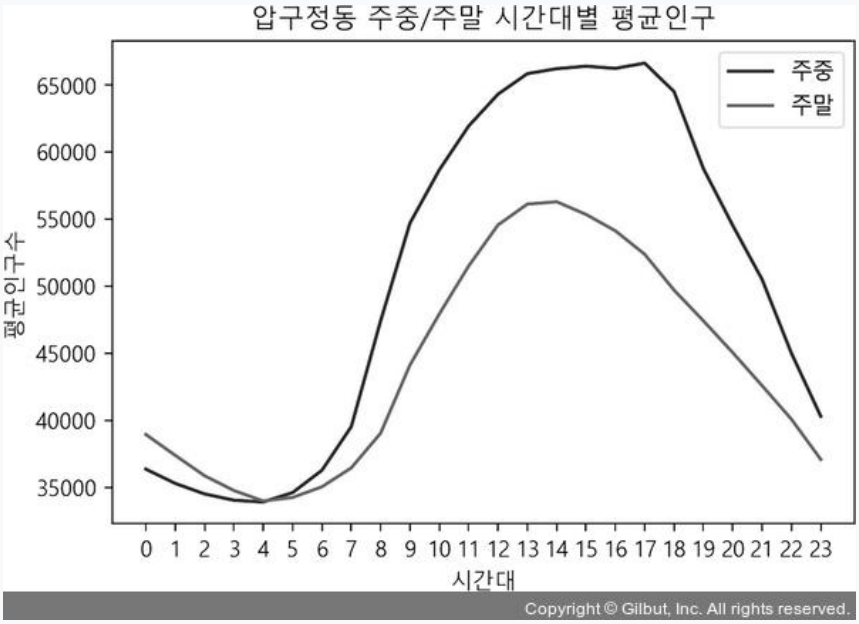
시간대별로 주중과 주말 인구가 조금씩 차이납니다.

마지막 과정 6도 작성해 봅시다. 주중과 주말 리스트를 각각 꺾은선 그래프로 그리고, 주중과 주말을 구분하기 위해 범례를 추가하고 그래프 색상도 다르게 합니다.

```
import matplotlib.pyplot as plt

plt.rc('font', family='Malgun Gothic')
plt.title(dong_name + ' 주중/주말 시간대별 평균인구')
plt.plot(weekday, color='indigo', label='주중')
plt.plot(weekend, color='orangered', label='주말')
plt.legend()
plt.xlabel('시간대')
plt.ylabel('평균인구수')
plt.xticks(range(24), range(24))
plt.show()
```

실행결과

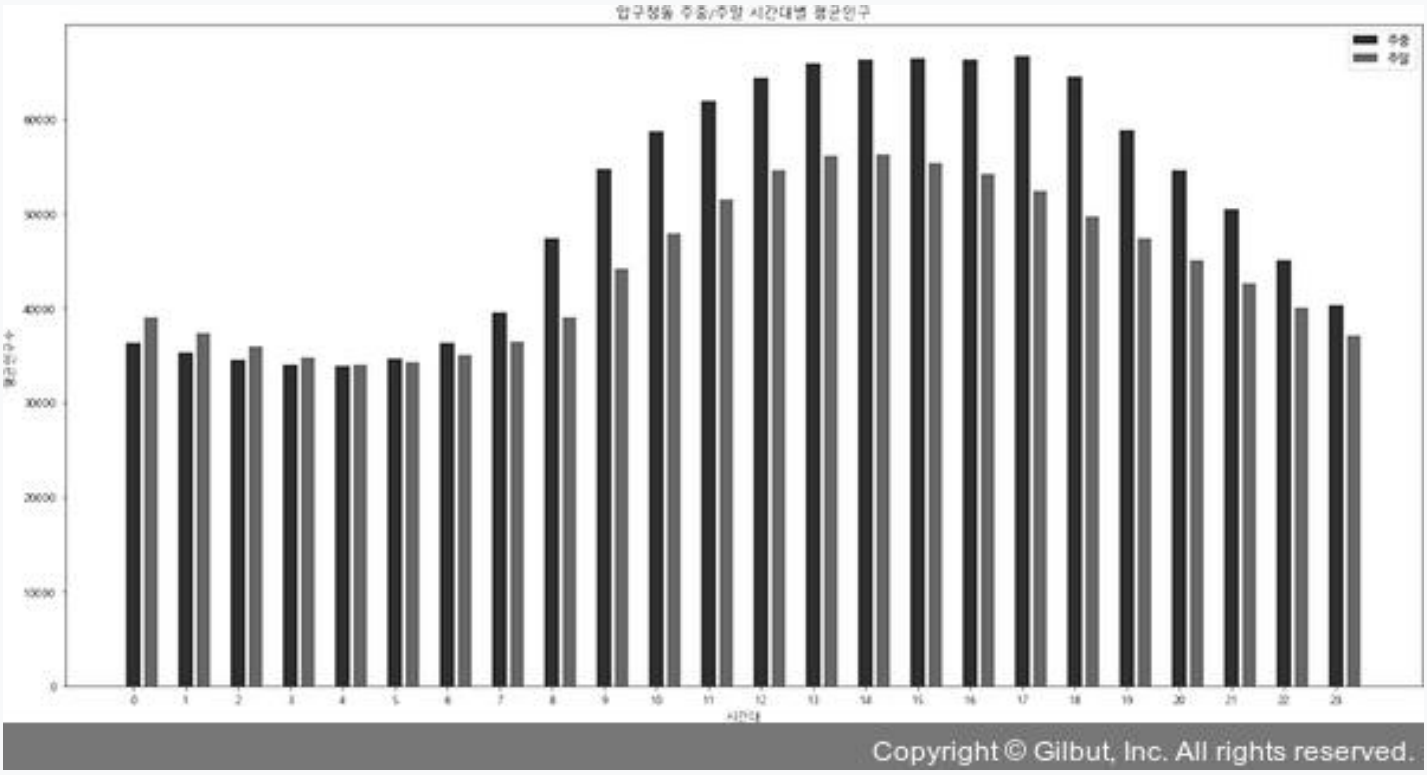


주중과 주말 인구의 꺾은선 그래프가 겹쳐 그려집니다.

하위 목표를 세울 때 막대그래프도 제시했죠? 막대그래프도 한번 그려 봅시다. 그래프 그리는 명령어를 바꾸고 x축 눈금을 조정하면 됩니다.

```
plt.figure(figsize=(20, 10))
plt.rc('font', family='Malgun Gothic')
plt.title(dong_name+ ' 주중/주말 시간대별 평균인구')
plt.bar(range(0, 72, 3), weekday, color='indigo', label='주중')
plt.bar(range(1, 73, 3), weekend, color='orangered', label='주말')
plt.legend()
plt.xlabel('시간대')
plt.ylabel('평균인구수')
plt.xticks(range(0, 72, 3), range(24))
plt.show()
```

실행결과



그래프 크기가 작으면 구분이 잘 안 될 수 있어서 `plt.figure()`로 그래프 크기를 키웠습니다. 여기서는 가로 크기를 20인치, 세로 크기를 10인치로 지정했습니다.

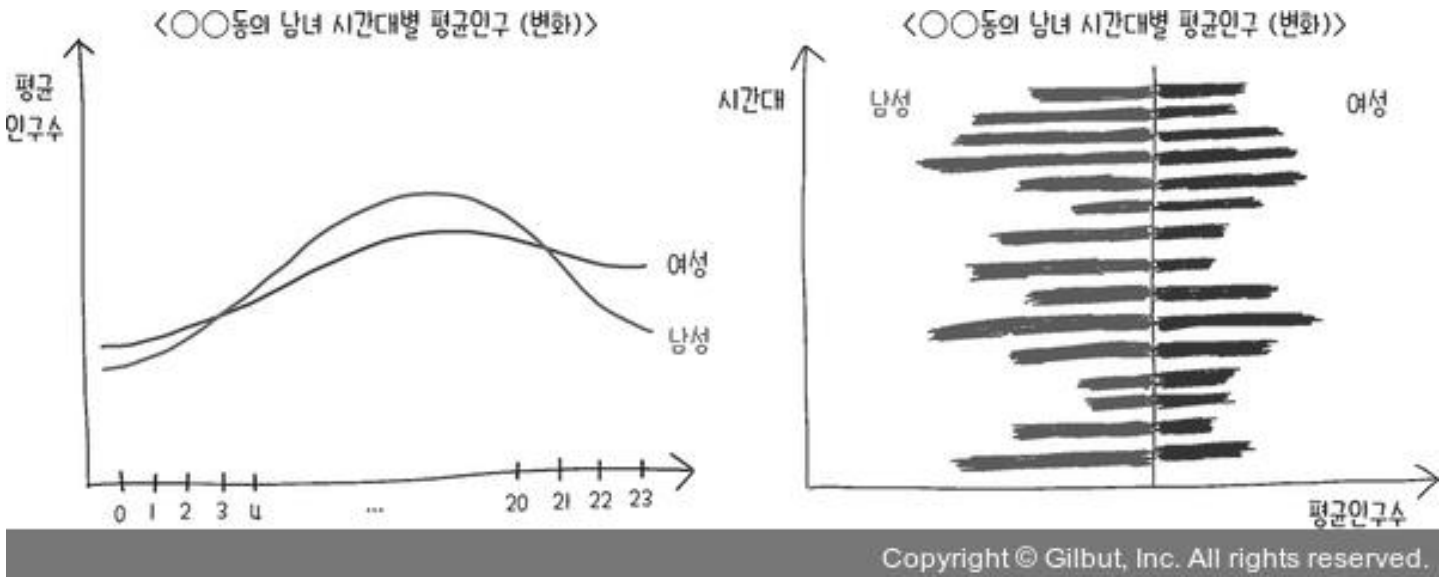
결과 분석하기
압구정에는 맛집이나 예쁜 카페가 많아서 당연히 주중보다는 주말에 사람이 더 많을 거라 생각했는데 예상과는 정반대의 결과가 나왔습니다. 대체로 주말보다 주중 인구가 훨씬 더 많군요. 주말에 압구정에 간다면 그리 걱정하지 않아도 되겠어요. 예외적으로 0시에서 4시 사이에 주중 인구보다 주말 인구가 더 많은 것으로 나오네요. 아마 유흥을 즐기는 인구가 아닐까 추측해 볼 수 있습니다.

11.2.4 하위 목표 3 - 남녀 시간대별 평균인구 분석하기

세 번째 하위 목표를 구현할 차례입니다. 목표 수립 단계에서 '핫플레이스가 있는 행정동의 남녀 시간대별 평균인구 그래프를 그려 분석하기'로 하위 목표를 잡았습니다.

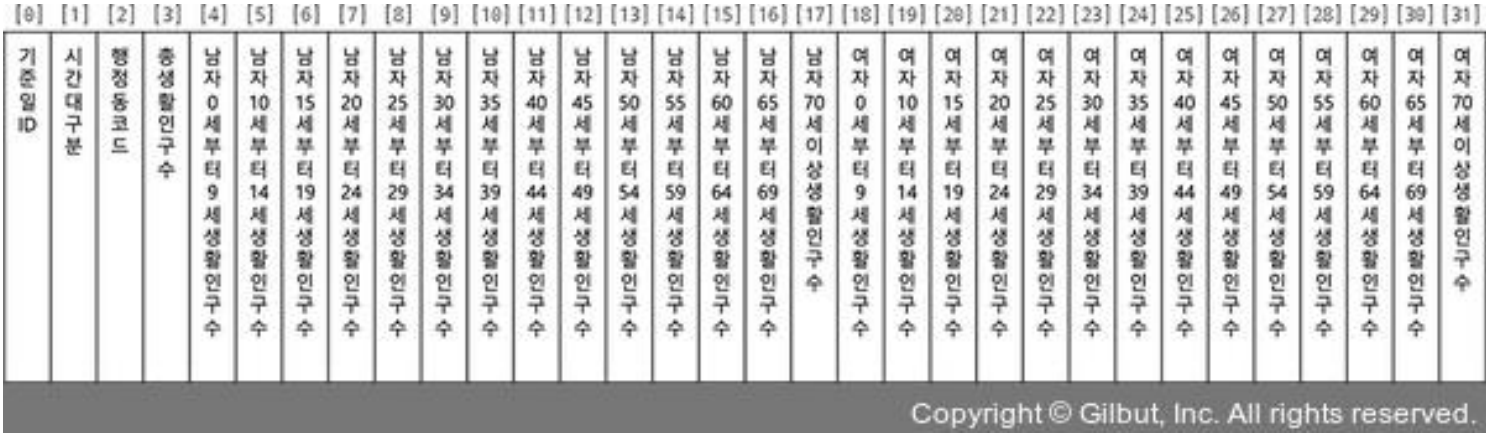
하위 목표 3

> 핫플레이스가 있는 행정동의 남녀 시간대별 평균인구 그래프를 그려 분석하기



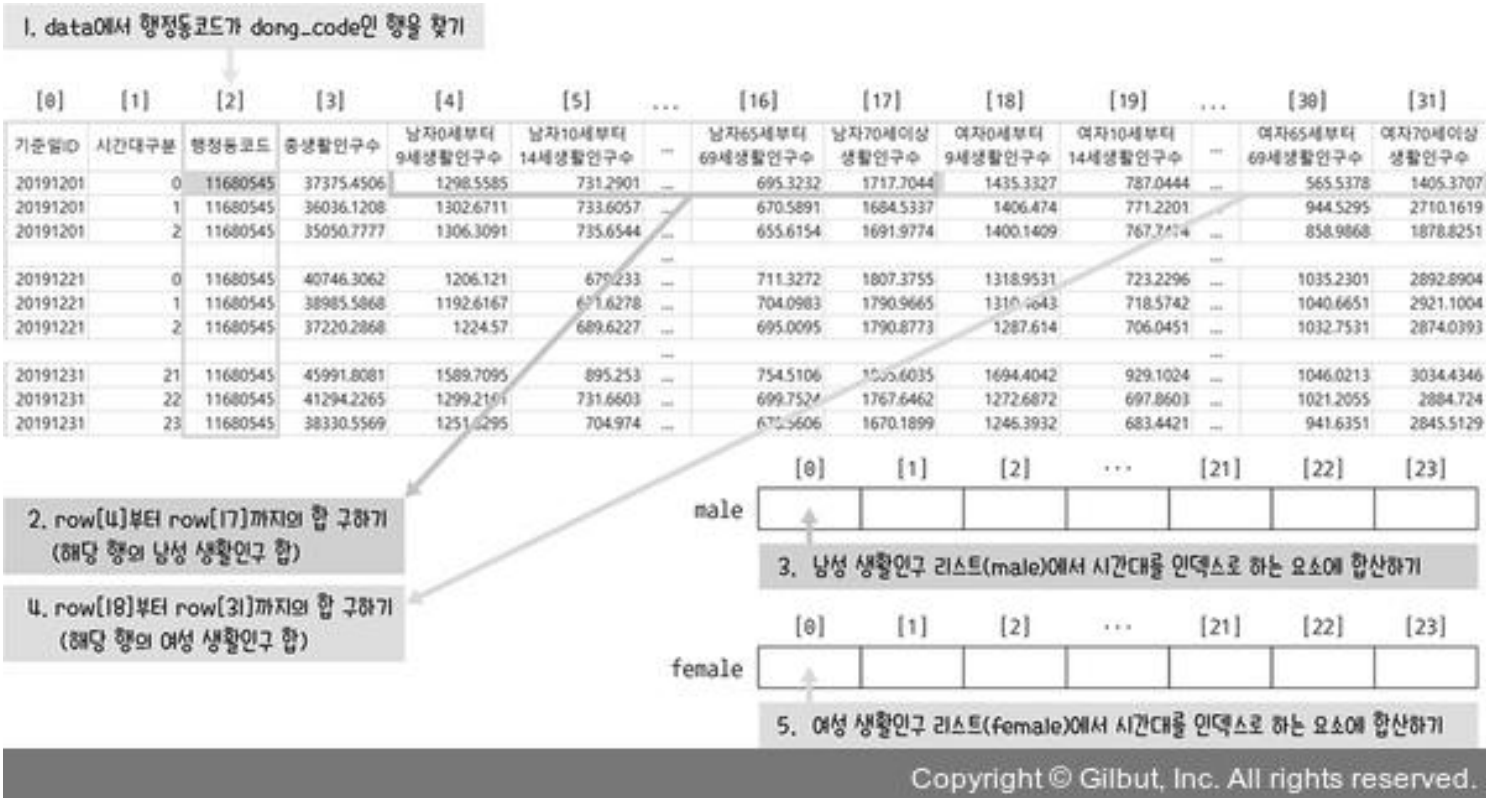
알고리즘 작성하기

주중과 주말처럼 남성과 여성의 시간대별 평균인구 그래프를 겹쳐 그려 비교하면 됩니다. 여기에서도 역시 리스트가 2개 필요합니다. 남성의 시간대별 평균인구를 담은 24칸짜리 리스트와 여성의 시간대별 평균인구를 담은 24칸짜리 리스트입니다. 남성 생활인구 리스트의 이름은 `male`, 여성 생활인구 리스트의 이름은 `female`로 하겠습니다. 성별은 주중/주말과 달리 데이터에 이미 구분되어 있어서 작업하기 쉽습니다.



▲ 그림 11-18 인구 데이터의 헤더와 인덱스

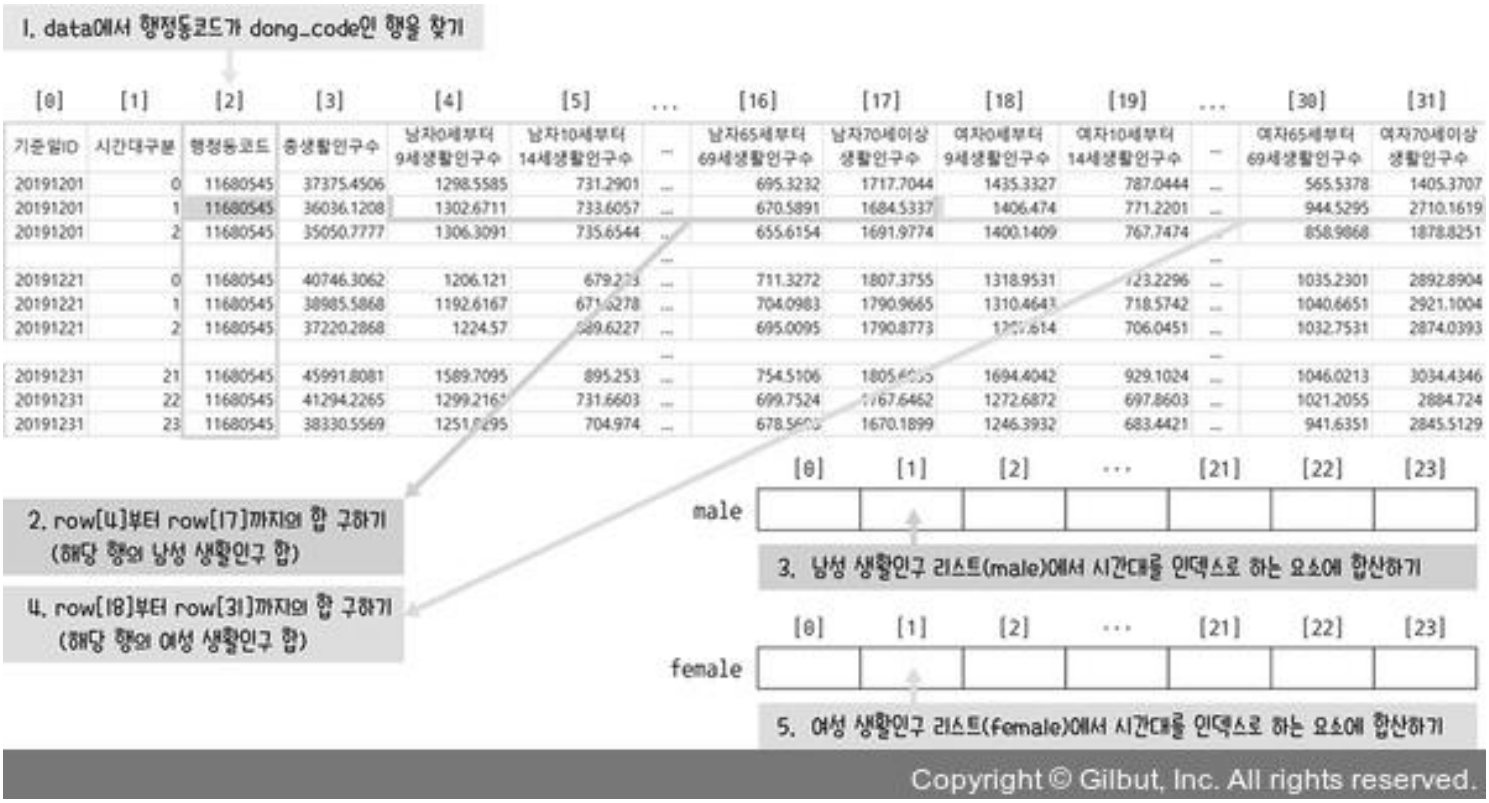
`data`의 열 인덱스 [4]부터 [17]까지 남성 생활인구 데이터가, [18]부터 [31]까지는 여성 생활인구 데이터가 들어 있습니다. 행정동코드가 11680545(압구정동)인 행을 골라낸 후 열 인덱스 [4]부터 [17]까지 들어 있는 값을 합하면 해당 행의 총 남성 생활인구가 됩니다. 열 인덱스 [18]부터 [31]까지 합을 구하면 해당 행의 총 여성 생활인구가 되죠. 즉, 행마다 남성 생활인구와 여성 생활인구의 합이 나오며 이 값을 성별에 맞는 리스트에 각각 더하면 됩니다. 여기에서도 각 행의 시간대가 곧 리스트의 인덱스입니다. 그림으로 정리해 보겠습니다. 번호 순서대로 흐름을 확인하세요.



▲ 그림 11-19 2019년 12월 1일 0시의 남녀 생활인구 구하기

그림은 `data`에서 행정동코드가 11680545(압구정동)인 행만 보여 줍니다. `data`가 날짜와 시간순으로 되어 있으므로 2019년 12월 1일 0시 데이터가 가장 먼저 선택됩니다. 첫 번째 행에서 열 인덱스 [4]~[17]의 합이 2019년 12월 1일 0시의 남성 생활인구가 되죠. `row[4] + row[5] + ... + row[17]`로 표현할 수 있습니다. 이 값을 `male[0]`에 더합니다. 열 인덱스 [18]~[31]의 합은 2019년 12월 1일의 0시의 여성 생활인구입니다. `row[18] + row[19] + ... + row[31]`로 합을 구하고 합한 값을 `female[0]`에 더합니다. 이렇게 하면 2019년 12월 1일 0시의 남성과 여성 생활인구가 각각 구해집니다.

두 번째 행도 봅시다. 과정은 동일합니다. 2019년 12월 1일 1시 데이터가 `row`에 들어 있다는 점만 다를 뿐이죠. 마찬가지로 열 인덱스 [4]부터 [17]까지의 합을 구한 후 `male[1]`에 더합니다. 열 인덱스 [18]부터 [31]까지의 합은 `female[1]`에 더하고요.



▲ 그림 11-20 2019년 12월 1일 1시의 남녀 생활인구 구하기

이 과정을 `data`의 행정동코드가 11680545인 모든 행에서 반복합니다. 그러면 가장 마지막 행은 2019년 12월 31일 23시 데이터가 됩니다. 마지막 행까지 반복하고 나면 남성과 여성 생활인구의 합이 시간대별로 구해집니다. 이때 `male`과 `female` 리스트 역시 한 달간의 생활인구 총합을 저장하고 있으므로 평균인구를 알려면 31로 나누어 저장하는 과정이 필요합니다. 그리고 구한 리스트로 마지막에 그래프를 그립니다.

하위 목표 3

핫플레이스가 있는 행정동의 남녀 시간대별 평균인구 그래프를 그려 분석하기

하위 목표 3

1. 남성 시간대별 생활인구를 저장할 리스트(male)와 여성 시간대별 생활인구를 저장할 리스트(female)를 길이 24로 만들고 초깃값 0 저장하기
2. 인구 데이터(data)의 첫 행부터 마지막 행까지 반복하기
    2.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
        2.1.1 해당 행의 시간대(열 인덱스 [1])를 변수(time)에 저장하기
        2.1.2 열 인덱스 [4]부터 [17]까지의 합을 male[time]에 더하기
        2.1.3 열 인덱스 [18]부터 [31]까지의 합을 female[time]에 더하기
3. 반복이 끝나면 남성 생활인구 리스트(male)와 여성 생활인구 리스트(female)를 각각 31로 나누어 시간대별 평균인구 구하기
4. 완성된 male과 female 리스트로 시간대별 평균인구 그래프 그리기

TIP '남성 생활인구를 구한 후 총생활인구에서 남성 생활인구를 빼면 여성 생활인구를 구할 수 있지 않을까' 하고 생각할 수 있습니다. 그런데 그렇게 하지 않은 이유가 있습니다. 인구 데이터를 내려받은 서울 열린데이터 광장 페이지를 보면 '추정된 데이터이기 때문에 각 값의 합은 전체합계와 일치하지 않을 수 있다'는 유의사항 있습니다. 이는 열 인덱스 [4]부터 [31]까지의 합이 해당 열의 총생활인구와 일치하지 않을 수 있다는 얘기입니다. 실제로 계산해 보면 인구 오차는 한 자리나 그 이하 소수점으로 크지 않습니다. 전체 통계에 큰 영향을 미치지 않는 수준이죠. 이런 사항을 인지하고 있다면 여성 생활인구를 총생활인구 - 남성 인구(`f = float(row[3]) - m`)로 계산해도 됩니다. 다만, 정확하지 않은 숫자이니 중요한 데이터를 분석할 때는 유의해야 합니다.

코드 작성하기

알고리즘을 코드로 구현해 보겠습니다.

〈알고리즘〉

1. 남성 시간대별 생활인구를 저장할 리스트(male)와 여성 시간대별 생활인구를 저장할 리스트(female)를 길이 24로 만들고 초깃값 0 저장하기
2. 인구 데이터(data)의 첫 행부터 마지막 행까지 반복하기
   2.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
      2.1.1 해당 행의 시간대(열 인덱스 [1])를 변수(time)에 저장하기
      2.1.2 열 인덱스 [4]부터 [17]까지의 합을 male[time]에 더하기
      2.1.3 열 인덱스 [18]부터 [31]까지의 합을 female[time]에 더하기
3. 반복이 끝나면 남성 생활인구 리스트(male)와 여성 생활인구 리스트(female)를 각각 31로 나누어 시간대별 평균인구 구하기
4. 완성된 male과 female 리스트로 시간대별 평균인구 그래프 그리기

〈코드〉

```
1. male = [0 for i in range(24)]
   female = [0 for i in range(24)]
2. for row in data:
    2.1 if row[2] == dong_code:
        2.1.1 time = row[1]
        2.1.2 male[time] += sum(row[4:18])
        2.1.3 female[time] += sum(row[18:32])
3. male = [m/31 for m in male]
   female = [f/31 for f in female]
4. plt.plot(range(24), male)
   plt.plot(range(24), female)
```

과정 2.1.1까지는 하위 목표 2와 비슷하므로 과정 2.1.2부터 살펴보겠습니다. 남성 생활인구는 `row[4] + row[5] + … + row[17]`로 구하는데, 이를 리스트 슬라이싱과 리스트 요소의 합을 구하는 `sum()`을 활용하면 간단해집니다(6.1.6 다양한 리스트 명령어 사용하기 참고). 먼저 `row[4:18]`로 리스트를 슬라이싱해 `row[4]`부터 `row[17]`까지 잘라냅니다. 이렇게 슬라이싱한 결과도 리스트이므로 `sum()`으로 요소의 합을 구합니다. `sum(row[4:18])`으로 해당 행의 남성 생활인구의 합을 구한 후 시간대별 남성 총생활인구인 `male[time]`에 합산합니다. 여성도 동일한 방법으로 `sum(row[18:32])`로 해당 행의 여성 생활인구의 합을 구한 후 시간대별 여성 총생활인구인 `female[time]`에 합산합니다.

성별 생활인구의 합이 모두 더해지면 과정 3에서 `male` 리스트와 `female` 리스트의 모든 요소를 31로 나누어 평균인구를 구합니다.

앞에서 작성한 코드를 프로그램으로 옮겨 봅시다. 전체 과정을 모두 작성합니다. 중간에 리스트에 담긴 값을 확인하고 싶다면 반복문 내부에 `print()`를 적절히 넣어 값을 확인해 보세요. 남성 평균인구 그래프와 여성 평균인구 그래프는 색을 다르게 해서 구분합니다.

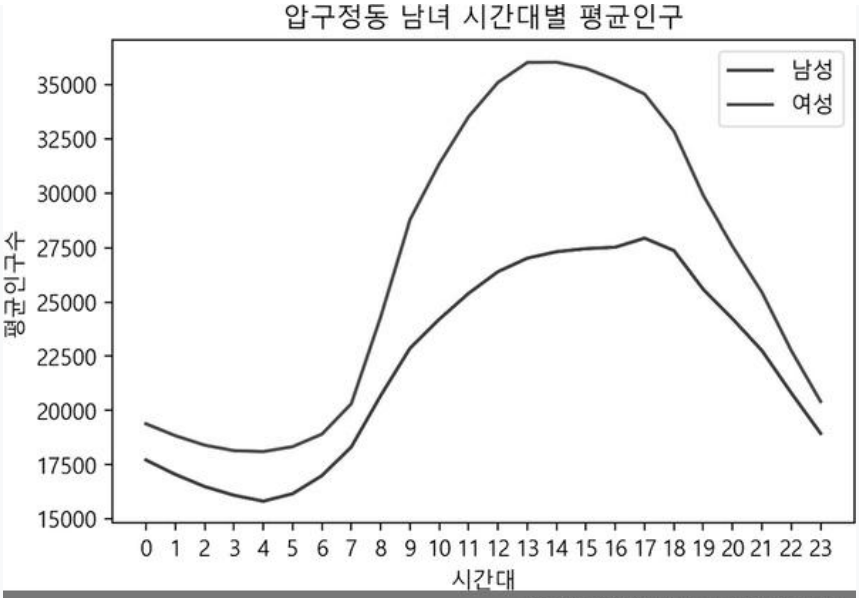
```
import matplotlib.pyplot as plt

male = [0 for i in range(24)]
female = [0 for i in range(24)]

for row in data:
    if row[2] == dong_code:
        time = row[1]
        male[time] += sum(row[4:18])      # 남성 인구 총합 구하기
        female[time] += sum(row[18:32])   # 여성 인구 총합 구하기
male = [m/31 for m in male]
female = [f/31 for f in female]

plt.rc('font', family='Malgun Gothic')
plt.title(dong_name + ' 남녀 시간대별 평균인구')
plt.plot(male, color='b', label='남성')
plt.plot(female, color='r', label='여성')
plt.xlabel('시간대')
plt.ylabel('평균인구수')
plt.legend()
plt.xticks(range(24), range(24))
plt.show()
```

실행결과



Copyright © Gilbut, Inc. All rights reserved.

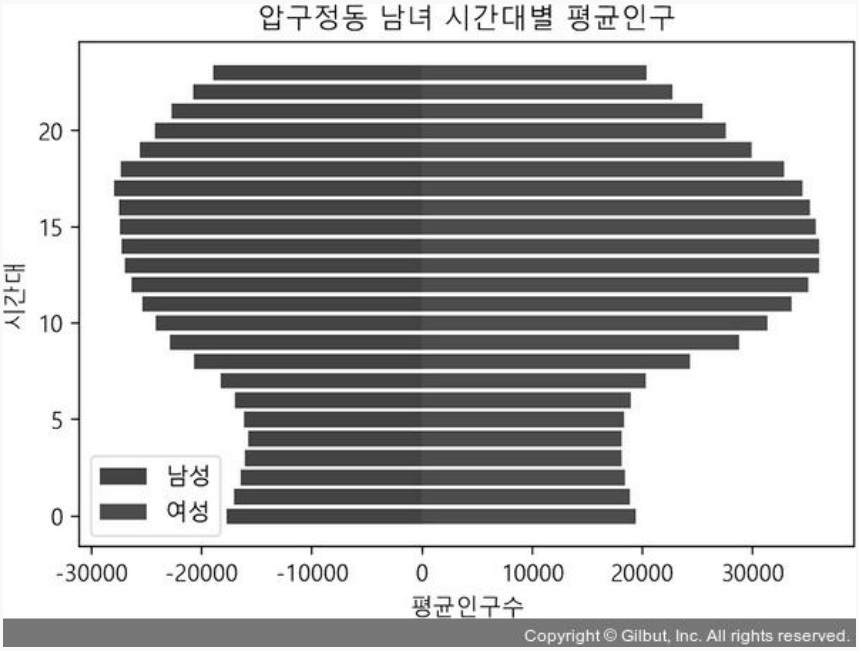
비교 그래프가 잘 그려집니다. 목표 수립 단계에서 스케치한 두 번째 그래프도 그려 봅시다. 좌우로 펼쳐진 가로형 막대그래프였죠. 남성 평균인구를 왼쪽에 여성 평균인구를 오른쪽에 표현해 보겠습니다. 그래프를 왼쪽에 표현하려면 데이터를 음수 처리해야 합니다. 적용해서 그려 봅시다.

```
import matplotlib

male = [-m for m in male]
matplotlib.rcParams['axes.unicode_minus'] = False

plt.rc('font', family='Malgun Gothic')
plt.title(dong_name + ' 남녀 시간대별 평균인구')
plt.barh(range(24), male, color='b', label ='남성')
plt.barh(range(24), female, color='r', label='여성')
plt.xlabel('평균인구수')
plt.ylabel('시간대')
plt.legend()
plt.show()
```

실행결과



왼쪽에 남성 데이터를 표시하기 위해 `male`의 모든 요소를 음수로 바꾸었습니다. 리스트 내포를 사용하면 반복문을 사용하면서도 간결하게 모든 요소를 음수로 바꿀 수 있습니다.

마이너스를 그래프에 표시하는 `matplotlib.rcParams['axes.unicode_minus'] = False` 코드도 잊지 마세요. 그리고 꺾은선 그래프와 달리 y축이 시간대, x축이 평균인구수이므로 `plt.xlabel()`과 `plt.ylabel()`도 바꿔야 합니다. y축이 시간대이므로 가장 작은 숫자인 0이 가장 밑에, 가장 큰 숫자인 23이 가장 위에 있습니다.

결과 분석하기

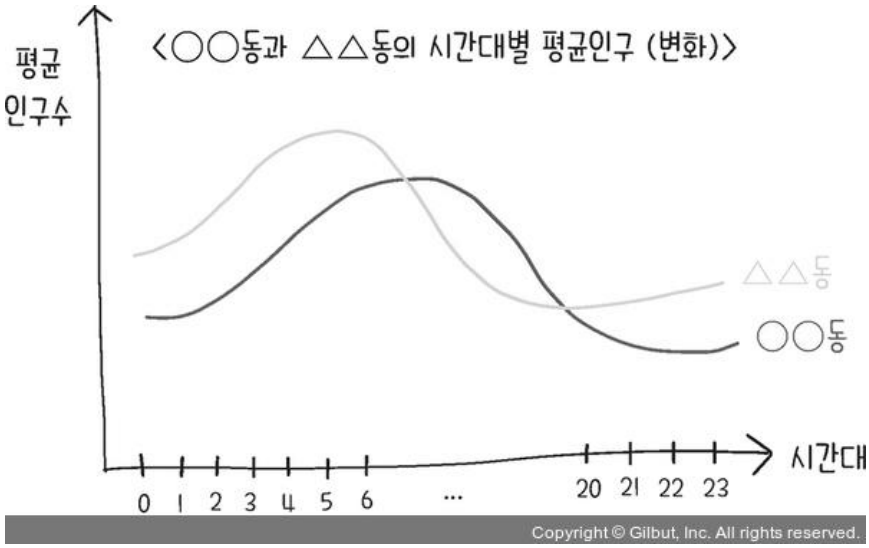
압구정동에는 여성이 남성보다 항상 더 많은 것으로 나타납니다. 남성보다 여성 인구가 출근 시간에 급격하게 늘어나고 퇴근 시간 이후 급격히 줄어듭니다. 물론 남성 인구의 증감 패턴도 비슷합니다. 하지만 그 증가폭과 감소폭이 여성보다는 훨씬 덜하죠. 아무래도 압구정동으로 출퇴근하는 여성이 남성보다 더 많은 것 같습니다. 그러니 만약 가고 싶은 핫플레이스가 여성이 선호하는 장소라면 점심 시간에 붐빌 수도 있겠네요. 막대그래프에서 그래프의 부피를 비교해봐도 여성 평균인구가 더 많은 것을 알 수 있습니다. 평소에 막연하게 느끼던 것을 그래프로 시각화하니 확실히 비교할 수 있습니다.

11.2.5 하위 목표 4 - 다른 지역과 인구 비교 분석하기

마지막 하위 목표를 작성해 봅시다. 지금까지는 인구의 절대치로만 그래프를 그렸기 때문에 실제로 어느 정도 붐비는지 실감나지 않습니다. 이를 위해 잘 아는 지역과 핫플레이스가 있는 지역의 인구 그래프를 비교해 보겠습니다. 목표 수립 단계에서 '핫플레이스가 있는 행정동과 익숙한 행정동의 시간대별 평균인구 그래프를 그려 비교 분석하기'로 하위 목표를 잡았습니다.

하위 목표 4

핫플레이스가 있는 행정동과 익숙한 행정동의 시간대별 평균인구 그래프를 그려 비교 분석하기



알고리즘 작성하기

그래프 2개를 겹쳐 그려야 하므로 역시 리스트가 2개 필요합니다. 두 지역의 시간대별 평균인구로 그래프를 그려 봅시다. 그런데 핫플레이스가 있는 행정동의 평균인구를 구하는 것은 이미 하위 목표 1에서 진행했습니다. 그러므로 여기에서는 잘 아는 지역을 입력받아 평균인구를 구하는 것만 추가로 하면 됩니다. 이 역시 어렵지 않습니다. 평균인구를 구하는 방법은 동일하기 때문이죠.

다만, 두 데이터가 겹치지 않게 서로 다른 변수를 사용해야 합니다. 핫플레이스가 있는 행정동코드와 인구 데이터는 `dong_code`와 `population`에, 비교할 지역의 행정동코드와 인구 데이터는 `dong_code2`와 `population2`에 저장합시다.

그리고 앞서 구현한 행정동명과 행정동코드 연결하기 코드를 두 번 반복해 한 번은 핫플레이스의 행정동을 입력받고(`dong_name`, `dong_code` 사용), 다른 한 번은 비교할 지역의 행정동을 입력받습니다(`dong_name2`, `dong_code2` 사용). 반복문으로 data의 첫 행부터 마지막 행까지 돌면서 행정동코드가 `dong_code`인 행의 총생활인구수를 `population`에, 행정동코드가 `dong_code2`인 행의 총생활인구수를 `population2`에 더합니다.

알고리즘으로 정리하면 다음과 같습니다.

하위 목표 4

핫플레이스가 있는 행정동과 익숙한 행정동의 시간대별 평균인구 그래프를 그려 비교 분석하기

알고리즘

1. 사용자에게서 행정동명을 입력받아 행정동코드를 구한 후 이 값을 dong_code에 저장하기
2. 비교할 행정동명을 입력받아 행정동코드를 구한 후 이 값을 dong_code2에 저장하기
3. 핫플레이스의 행정동 인구를 저장할 리스트(population)와 비교할 행정동 인구를 저장할 리스트(population2)를 길이 24로 만들고 초깃값 0 저장하기
4. 인구 데이터(data)의 첫 행부터 마지막 행까지 반복하기
   4.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
      4.1.1 해당 행의 시간대(열 인덱스 [1])와 총생활인구수(열 인덱스 [3])를 각각 변수(time, p)에 저장하기
      4.1.2 인덱스가 time인 population 리스트의 요소(population[time])에 총생활인구수(p)를 더하기
   4.2 비교할 행정동의 행정동코드(dong_code2)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
      4.2.1 해당 행의 시간대(열 인덱스[1])와 총생활인구수(열 인덱스[3])를 각각 변수(time, p)에 저장하기
      4.2.2 인덱스가 time인 population2 리스트의 요소(population2[time])에 총생활인구수(p)를 더하기
5. 반복이 끝나면 두 리스트(population, population2)의 모든 값을 31로 나누어 다시 저장하기
6. 완성된 리스트로 두 지역의 시간대별 평균인구 그래프 그리기

코드 작성하기

동일한 원리가 반복되는 구조라 코드로 구현하기 어렵진 않습니다. 각 알고리즘 과정에 해당하는 코드를 작성해 봅시다.

> 〈알고리즘〉
>
> 1. 사용자에게서 행정동명을 입력받아 행정동코드를 구한 후 이 값을 dong_code에 저장하기
> 2. 비교할 행정동명을 입력받아 행정동코드를 구한 후 이 값을 dong_code2에 저장하기
> 3. 핫플레이스의 행정동 인구를 저장할 리스트(population)와 비교할 행정동 인구를 저장할 리스트(population2)를 길이 24로 만들고 초깃값 0 저장하기
> 4. 인구 데이터(data)의 첫 행부터 마지막 행까지 반복하기
>   4.1 사용자가 입력한 행정동의 행정동코드(dong_code)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
>   4.1.1 해당 행의 시간대(열 인덱스 [1])와 총생활인구수(열 인덱스 [3])를 각각 변수(time, p)에 저장하기
>   4.1.2 인덱스가 time인 population 리스트의 요소(population[time])에 총생활인구수(p)를 더하기
>   4.2 비교할 행정동의 행정동코드(dong_code2)와 인구 데이터(data)의 행정동코드(열 인덱스 [2])가 같다면
>   4.2.1 해당 행의 시간대(열 인덱스[1])와 총생활인구수(열 인덱스[3])를 각각 변수(time, p)에 저장하기
>   4.2.2 인덱스가 time인 population2 리스트의 요소(population2[time])에 총생활인구수(p)를 더하기
> 5. 반복이 끝나면 두 리스트(population, population2)의 모든 값을 31로 나누어 다시 저장하기
> 6. 완성된 리스트로 두 지역의 시간대별 평균인구 그래프 그리기
>
> ---

〈코드〉

```
1. 행정동명과 행정동코드 연결하기 코드 재사용
2. 행정동명과 행정동코드 연결하기 코드 재사용
3. population = [0 for i in range(24)]
   population2 = [0 for i in range(24)]
4. for row in data:
    4.1 if row[2] == dong_code:
        4.1.1 time, p = row[1], row[3]
        4.1.2 population[time] += p
    4.2 elif row[2] == dong_code2 :
        4.2.1 time, p = row[1], row[3]
        4.2.2 population2[time] += p
5. population = [p/31 for p in population]
   population2 = [p/31 for p in population2]
6. plt.plot(population)
   plt.plot(population2)
```

과정 1과 2만 프로그램으로 옮겨 봅시다.

```
# 핫플레이스가 있는 행정동
dong_name = input('핫플레이스가 위치한 행정동을 입력하세요. --> ')
for row in code_data:
    if row[-1] == dong_name:
        dong_code = row[1]

# 비교할 행정동
dong_name2 = input('비교할 행정동을 입력하세요. --> ')
for row in code_data:
    if row[-1] == dong_name2:
        dong_code2 = row[1]
```

핫플레이스가 있는 행정동은 '압구정동', 비교할 행정동은 '역삼2동'으로 입력해 실행합니다.

```
실행결과
핫플레이스가 위치한 행정동을 입력하세요. --> 압구정동
비교할 행정동을 입력하세요. --> 역삼2동
```

앞에서 설명한 대로 행정동명과 행정동코드 연결하기 코드를 두 번 반복하되 변수만 다르게 넣습니다.

과정 4는 `data`를 반복문으로 돌며 반복문 내부에서 `data`의 행정동코드(`row[2]`)가 `dong_code`와 같은지, `dong_code2`와 같은지를 `if-elif` 문으로 판단합니다. 그리고 조건에 맞는 시간대별 총생활인구(`p`)를 인덱스(`[time]`)에 맞게 저장하도록 코딩합니다. 나머지 과정은 기존 코드와 거의 같습니다. 그래프는 그래프 제목과 x축 이름, y축 이름만 바꿔 그리면 됩니다.

그럼 나머지 과정을 프로그램으로 옮겨 봅시다.

```
import matplotlib.pyplot as plt

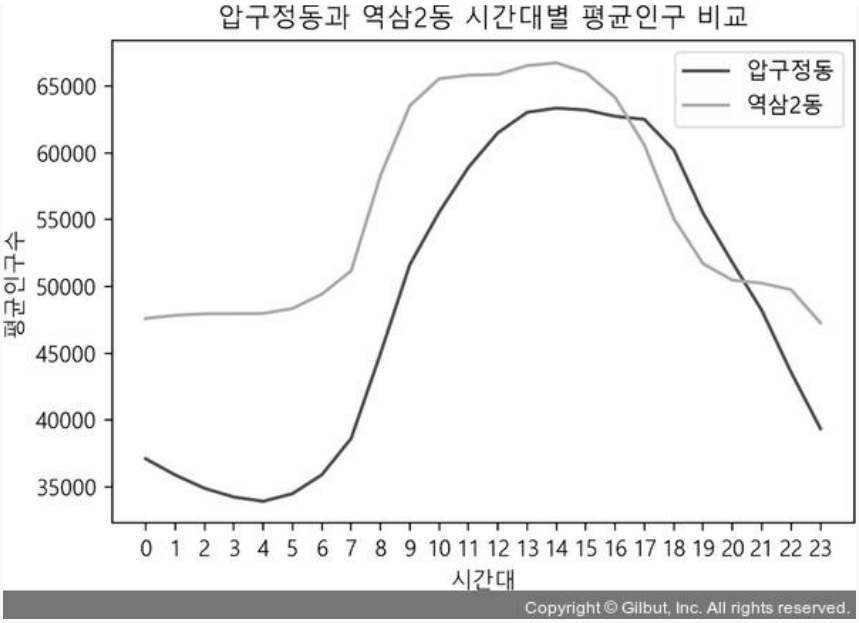
population = [0 for i in range(24)]
population2 = [0 for i in range(24)]

for row in data:
    # 핫플레이스가 있는 행정동인 경우
    if row[2] == dong_code:
        time, p = row[1], row[3]
        population[time] += p
    # 비교할 지역의 행정동인 경우
    elif row[2] == dong_code2:
        time, p = row[1], row[3]
        population2[time] += p

population = [p/31 for p in population]
population2 = [p/31 for p in population2]

plt.rc('font', family='Malgun Gothic')
plt.title(dong_name + '과 ' + dong_name2 + ' 시간대별 평균인구 비교')
plt.plot(population, color='m', label=dong_name)
plt.plot(population2, color='orange', label=dong_name2)
plt.legend()
plt.xlabel('시간대')
plt.ylabel('평균인구수')
plt.xticks(range(24), range(24))
plt.show()
```

실행결과



그래프가 그려졌으니 결과를 분석하는 시간을 가져 볼까요?

결과 분석하기

역삼2동은 새벽에 인구수가 4만 7천~4만 8천 명 정도인데 압구정동은 3만 5천 명 내외네요. 새벽에는 인구 이동이 적으므로 새벽 인구가 해당 동에 거주하는 인구라 가정한다면, 역삼2동이 압구정동보다 거주 인구가 많은 것으로 보입니다.

그런데 인구 이동은 역삼2동보다 압구정동이 더 많은 것으로 나타나죠. 압구정동의 등락폭은 약 2만 9천 명이고 역삼2동의 등락폭은 약 2만 명입니다. 대부분의 시간에는 역삼2동이 인구가 더 많지만, 17~20시 사이에는 역전되네요. 이 시간대에 압구정동이 역삼2동보다 더 붐빌 것 같습니다. 이 시간대 외에는 압구정동으로 가는 것이 더 나을 것 같고요. 이처럼 잘 아는 지역을 직접 입력해 인구 그래프를 비교해 보세요. 막연하게 느낀 부분들이 그래프에 정확한 수치로 나타날 겁니다.

사실 두 행정동의 크기 차이까지 고려하면 붐비는 정도는 다를 수 있습니다. 여기서는 단순히 평균인구수로만 비교하고 있으니 참고해 주세요.